

Embryo Image Analysis

Final Report

Spring 2026

Project Team:

Name	Course	Word Count
Kyla Mae Lumabao	ECE 496	3,728
Jasmine Meissner	ECE 496	2823
Marcus Siarot	ECE 496	2709
Jeremy Molina	ECE 396	338
Nathaniel Alvaran	ECE 296	134

5/08/2026

Faculty Advisor: Dr. Aaron Ohta

Abstract:

This report describes how the Embryo Image Analysis team improved our Embryo Image Analysis application and developed detection systems for environmental safety. Our focus was to organize the application software better, train neural network models to identify the parts of the embryo, and implement LiDAR technology to detect and locate unexploded ordnance (UXO).

For the application software, the original software code was contained in one large file, making it difficult to make updates. Our solution was to break the code into smaller, separate files for the user interface, video analysis, and results. The application became much easier to manage after doing this. The user interface was updated to look more appealing and respond better to user input. The app also now has a progress bar and can analyze multiple uploaded videos; before this semester, only one video could be uploaded and analyzed at a time.

For the neural networks, we replaced the older ResUNet-50 model with a more advanced framework called nnUNet. This new model outperformed the previous model – which we recorded Intersection over Union (IoU) values of 81.99% for Trophectoderm (TE) and 87.58% for Inner Cell Mass (ICM) as of Fall 2023 – achieving 99.78% and 99.91% respectively.. To further validate segmentation accuracy, the nnUNet model also achieved Dice coefficients of 99.89% for the TE and 99.96% for the ICM.

In addition to embryo analysis, the team also worked on a system to detect unexploded ordnance in complex environments. By using 3D LiDAR data, and architectures like Voxel R-CNN, the system can identify dangerous objects hidden in rocky and rough terrain or dense vegetation. The process involves converting 3D point clouds into voxels to help the neural network recognize the shapes and locations of UXOs. Through this work, we established a foundation for future training by identifying key components regarding dataset curation and neural network architectures.

Procedure:

Modular Application:

The previous codebase consisted of a single file for the entire application containing approximately 1,000 lines of code, making it difficult to trace errors, locate specific functionality, and avoid issues that could affect many different areas at once. The goal this semester was to decompose that entangled structure into clearly defined, standalone modules where each file is responsible for a single concern.

To achieve this project goal in making the application modular, we first had to understand how to organize code using classes, where we want to put them into their perspective files. A class is a user-defined data type that acts as a blueprint for creating objects. To get something usable out of a class, you create an instance of it, which is called an object. A helpful analogy is a floor plan for a house. The floor plan itself is not a house you can live in, but you can create as many houses from it as you want. In software, this means you can define a class once and instantiate it wherever it is needed, with each instance managing its own state independently. You can create as many objects from the same class as needed, and each one operates independently without interfering with others.

In Python, every class has a special method called `__init__`, known as the constructor, which Python runs automatically when an object is created. Its job is to set up the object's initial state, gathering everything it needs to function in one place rather than scattering setup code across the script. Inside every method, the first parameter is `self`, which refers to the specific object the method belongs to. Storing data as `self.variable` attaches it to that object alone, which is a fundamental advantage over global variables since every piece of data is always clear, making bugs easier to find and fixes safer to apply [1].

Classes can also build on top of one another through inheritance. A child class inherits all the behavior of its parent class and only needs to define what is new or different. We call `super().__init__()` at the start of the child's constructor to ensure that the parent class is properly initialized before the child class adds anything on top [1]. This is particularly useful in Graphical User Interface (GUI) frameworks, where inheriting from a built-in window or frame type instantly gives a class all of the framework's built-in functionality without reimplementing it from scratch.

Closely related to inheritance is polymorphism, which allows different classes to share the same method name while each providing their own implementation [1]. The correct version is called automatically based on which object it belongs to, meaning the rest of the application can interact with different components through a consistent interface without needing to know the internal details of each one.

Finally, splitting the codebase into multiple files is managed through Python's import system. Just as we import external libraries and modules such as `os`, `cv2`, and `CustomTkinter`, we also import our own files by name – for example, `from video_analysis import VideoAnalysis` – so that one file can access another file's classes, attributes, and methods as needed. Imports should flow in one direction to avoid circular imports, where two files import from each other and neither can finish loading. Data that needs to travel the other direction is instead passed as a reference through the constructor, keeping the import structure clean while still allowing full communication between components. This is beneficial for this project because it encapsulates data, promotes modularity, readability, and reusability, and ensures that functions do not depend on one another in hidden ways – making the codebase significantly easier for the team to navigate and maintain.

To begin the transition, we analyzed all parts of the existing code to determine how to best separate it into logical, self-contained files. We organized the code based on the order in which a user interacts with the application, resulting in the following six files: `main.py`, `app.py`, `main_menu.py`, `initial_conditions.py`, `video_analysis.py`, and `analyzed_results.py`. All of these files are located under one folder called `Sp26_Windows`. When running the application, the user runs `main.py`, since that is the entry point where the application begins.

First, `main.py` contains imports, sets up the application environment, and creates and runs the `App` object. It is the first file that runs when the application is launched. It first configures `OpenCV`, `Tensorflow`, `Pytesseract`, and the base path, then loads the `ResUNet-50` model so it is ready before the window opens, passing both the base path and the model into the `App` class. Figure 1, the code `if __name__ == "__main__"` guard ensures that an `App` object is only created when the file is run directly. Once created, `mainloop()` is called, which starts the `CTkinter` event loop and keeps the window open and responsive until the user closes it.

```

import multiprocessing
import sys
import os

# Limit CPU threads for TensorFlow and OpenCV
os.environ["OMP_NUM_THREADS"] = str(int(multiprocessing.cpu_count() / 2))

import tensorflow as tf
tf.config.threading.set_inter_op_parallelism_threads(1)
tf.config.threading.set_intra_op_parallelism_threads(1)

import pytesseract
from keras_segmentation import pretrained

from app import App

# Set Tesseract binary path (Windows)
pytesseract.pytesseract.tesseract_cmd = 'Tesseract-OCR/tesseract.exe'

# Set base path for PyInstaller compatibility
try:
    base_path = sys._MEIPASS
except Exception:
    base_path = os.path.abspath(".")

# Configure the model
model_config = {
    "input_height": 480,
    "input_width": 480,
    "n_classes": 3,
    "model_class": "resnet50_unet"
}

# Load model using the latest weights
latest_weights = os.path.join(base_path, '1911resnet50_fullbatch_80epochs.h5')
model = pretrained.model_from_checkpoint_path(model_config, latest_weights)

if __name__ == "__main__":
    app = App(base_path, model)
    app.mainloop()

```

Figure 1: main.py

In app.py it defines the App class, which is the root window of the entire application, as shown in Figure 2. It inherits from ctk.CTk, making it a fully functional CustomTkinter window. The constructor receives base_path and model from main.py and stores them as instance attributes so any other class holding a reference to app can access them. It then configures the window by setting the title, loading and applying the icon, setting the background color, enabling resizing, and enforcing a minimum window size of 1280x750 pixels. The minimum size was added to prevent the user from shrinking the window to the point where the content becomes invisible.

Once the window is set up, it creates the first MainMenu instance and packs it to fill the entire window, which is the initial screen the user sees.

```
import os
import customtkinter as ctk
import webbrowser

from main_menu import MainMenu

ctk.set_appearance_mode("light")

W = 1280 # window width
H = 750 # window height

class App(ctk.CTk):
    """The root window."""

    def __init__(self, base_path, model):
        super().__init__()
        self.base_path = base_path
        self.icon = os.path.join(base_path, 'embryoIcon2-1.ico')
        self.model = model

        # Setup window
        self.title("Embryo Analysis")
        self.iconbitmap(self.icon)
        self.configure(fg_color=█ "#f1f1f1")
        self.resizable(True, True)
        self.minsize(W, H)

        # Set window size, centers window on screen, allows resizing
        screen_width = self.winfo_screenwidth()
        screen_height = self.winfo_screenheight()
        x = (screen_width - W) // 2
        y = (screen_height - H) // 2
        self.geometry(f"{W}x{H}+{x}+{y}")

        # Initial Main Menu
        self.main_menu = MainMenu(self)
        self.main_menu.pack(fill="both", expand=True)
```

Figure 2: app.py constructor and attributes

We also created a method called `create_main_menu` and it exists for one specific case: when the user finishes viewing results and chooses to load new videos. It destroys the existing MainMenu instance and creates a fresh one, effectively resetting the application to its starting state without closing and reopening the window. The `open_help` method opens the application's help document in the user's default web browser, as shown in Figure 3. It is defined in `app.py` because the help URL is an application-wide resource that any screen can access by calling `app.open_help()`.

```

def create_main_menu(self):
    """
    Destroys previous main menu and creates a new one for new videos.
    Only used when user loads new videos after analyzing.
    """
    if self.main_menu and self.main_menu.winfo_exists():
        self.main_menu.destroy()
    self.main_menu = MainMenu(self)
    self.main_menu.pack(fill="both", expand=True)

def open_help(self):
    webbrowser.open('https://drive.google.com/file/d/1IT--74mgCW4EIjy1eQqzz98H19NdClUq/view?usp=sharing')

```

Figure 3: app.py methods

As for main_menu.py that defines the MainMenu class, which is the first screen the users interact with. It handles video uploading, initial condition management, and launching the analysis shown in Figure 4. When created, it initializes an initial_conditions dictionary that maps each uploaded video's file path to its embryo size and percent threshold values. This single dictionary replaces the three parallel global lists from the previous codebase. We added a navigation bar at the top of the screen containing a help button. Below that is the large bold "EMBRYO ANALYSIS" header. The main content area is the CTkListbox that displays the full file paths of all uploaded videos. When a user clicks on a video in the listbox, the `_update_conditions` method in Figure 5 will update the initial embryo size and percent threshold displayed below the listbox to reflect that video's current values, as shown in Figure 6. A fixed-height frame below the listbox uses `pack_propagate(False)` to prevent a container widget (like a frame) from resizing itself based on its contents. This frame holds two sections: the initial conditions displayed for the selected video and the four buttons – Upload, Remove, Edit Initial Conditions, and Analyze as shown in Figure 6. The progress bar and percentage completion label are also created here but are not yet displayed at this stage.

```

import customtkinter as ctk
from CTkListbox import CTkListbox
from modules.CTkMessageBox.ctkmessagebox import CTkMessageBox

from initial_conditions import InitialConditions
from analyzed_results import AnalyzedResults
from video_analysis import VideoAnalysis

class MainMenu(ctk.CTkFrame):
    """
    Main menu screen for uploading videos, setting
    conditions, and starting video analysis.
    """

    def __init__(self, app):
        super().__init__(app)
        self.app = app
        self.initial_conditions = {}

        # Setup window
        self.configure(fg_color="transparent")

        # Top navigation bar
        nav = ctk.CTkFrame(self, fg_color="transparent")
        nav.pack(fill="x", padx=10, pady=10)
        ctk.CTkButton(nav, width=40, text="?", command=app.open_help).pack(side="right")

        # Header
        ctk.CTkLabel(self, text="EMBRYO ANALYSIS", font=("Poppins", 48, "bold")).pack(side="top")

        # Listbox to display uploaded videos
        self.video_listbox = CTkListbox(self, font=("Poppins", 16), fg_color="■ #f5f5f5")
        self.video_listbox.pack(fill="both", expand=True, padx=150)
        self.video_listbox.bind("<<ListboxSelect>>", self._update_conditions)

        frame = ctk.CTkFrame(self, height=150, fg_color="transparent")
        frame.pack_propagate(False) # Prevents frame from resizing based on widgets inside
        frame.pack(fill="x", padx=150)

```

Figure 4: main_menu.py attributes


```

def _update_conditions(self, event=None):
    """
    Update the Embryo Size and Percent Threshold
    displayed when selecting a video
    """
    index = self._curselection()
    if index is None or self.video_listbox.size() == 0:
        self.embryo_size_value.configure(text="")
        self.percent_threshold_value.configure(text="")
        return

    video_path = self.video_listbox.get(index)

    if video_path not in self.initial_conditions:
        self.embryo_size_value.configure(text="")
        self.percent_threshold_value.configure(text="")
        return

    initial_conditions = self.initial_conditions[video_path]
    self.embryo_size_value.configure(text=initial_conditions["embryo_size"])
    self.percent_threshold_value.configure(text=initial_conditions["percent_threshold"])

```

Figure 5: `_update_conditions` method

```

# Initial Embryo Size / Percent Threshold display for selected video in Listbox
self.conditions = ctk.CTkFrame(frame, fg_color="transparent")
self.conditions.pack(fill="x")

embryo_size = ctk.CTkFrame(self.conditions, fg_color="transparent")
embryo_size.pack(side="left", padx=5)
ctk.CTkLabel(embryo_size, text="Initial Embryo Size ( $\mu\text{m}^2$ ) :", font=("Poppins", 10)).pack(side="left")
self.embryo_size_value = ctk.CTkLabel(embryo_size, text="", font=("Poppins", 10))
self.embryo_size_value.pack(side="left", padx=5)

percent_threshold = ctk.CTkFrame(self.conditions, fg_color="transparent")
percent_threshold.pack(side="left", padx=5)
ctk.CTkLabel(percent_threshold, text="Percent Threshold (%) :", font=("Poppins", 10)).pack(side="left")
self.percent_threshold_value = ctk.CTkLabel(percent_threshold, text="", font=("Poppins", 10))
self.percent_threshold_value.pack(side="left", padx=5)

# Buttons
self.buttons = ctk.CTkFrame(frame, fg_color="transparent")
self.buttons.pack(fill="x")

button_config = {"master": self.buttons, "width": 100, "height": 25}
ctk.CTkButton(**button_config, text="Edit Initial Conditions", command=self._edit_conditions).pack(side="left")
ctk.CTkButton(**button_config, text="Analyze", command=self._run_analysis).pack(side="right")
ctk.CTkButton(**button_config, text="Remove", command=self._remove_file).pack(side="right", padx=1)
ctk.CTkButton(**button_config, text="Upload", command=self._upload_videos).pack(side="right")

# Video Analysis Progress Bar
self.progress = ctk.CTkLabel(frame, text="", font=("Poppins", 12))
self.progressbar = ctk.CTkProgressBar(frame, orientation="horizontal", width=300, mode="determinate")
self.percentage = ctk.CTkLabel(frame, text="0% Completed", font=("Poppins", 12))

```

Figure 6: `main_menu.py` added attributes

In Figure 7, the original functions have been refactored into methods of the `MainMenu` class. Clicking Upload runs the `_upload_videos` function, which opens a file dialog filtered to

supported video formats and adds the selected files to the listbox, creating a default conditions entry for each one. Clicking Remove runs the function `_remove_file`, which deletes the currently selected video from both the listbox and the conditions dictionary. We removed the old `RemoveAllVideos()` and `removeTemporaryDirs()` functions shown in Figure 8, since their responsibilities could be consolidated – one handled removing videos from the UI and the other removed the associated directories, but together they covered the cleanup for a single video and were better expressed as one cohesive operation. Clicking Edit Initial Conditions runs the `_edit_conditions`, which opens the `InitialConditions` popup for the selected video. Clicking Analyze runs `_run_analysis`, which first validates that at least one video has been uploaded, then swaps the conditions display and buttons out for the progress bar in their place, then creates a `VideoAnalysis` instance and calls its `run()` method.

With the progress bar now visible during analysis, users can see that the application is actively processing their videos rather than appearing frozen while waiting. The progress bar value and percentage label update proportionally based on the number of videos being analyzed. Each button and its corresponding function are shown in Figures 7 and 9. When the analysis is complete, the progress bar is hidden, the main menu frame is unpacked, and an `AnalyzedResults` frame is packed in its place, transitioning the application to the results screen.

```

79
80     def _curselection(self):
81         """Return the current selected video index."""
82         index = self.video_listbox.curselection()
83         if index is None:
84             return None
85         if isinstance(index, tuple):
86             index = index[0]
87         return index
88
89
90     def _upload_videos(self):
91         """Add videos to the Listbox and generate default conditions"""
92         video_paths = ctk.filedialog.askopenfilenames(
93             filetypes=[("AVI", ".avi"), ("MOV", ".mov"), ("MP4", ".mp4"), ("FLV", ".flv")]
94         )
95         for path in video_paths:
96             self.video_listbox.insert("end", path)
97             self.initial_conditions[path] = {
98                 "embryo_size": "0",
99                 "percent_threshold": "0"
100             }
101
102
103     def _remove_file(self):
104         """Remove the selected file from the Listbox and initial conditions data."""
105         index = self._curselection()
106         if index is None:
107             return
108         video_path = self.video_listbox.get(index)
109         self.video_listbox.delete(index)
110         self.initial_conditions.pop(video_path)
111         self._update_conditions()
112
113
114     def _edit_conditions(self):
115         """Open the Edit Initial Conditions window for the selected video."""
116         index = self._curselection()
117         if index is None:
118             return
119         video_path = self.video_listbox.get(index)
120         InitialConditions(self.app, video_path)

```

Figure 7: Optimized original functions (methods)

```

## Define event: Open file explorer, select video, display selected video in table
def uploadVideos():
    filepath = tk.filedialog.askopenfilenames(filetypes=[("AVI", ".avi"),
        ("MOV", ".mov"), ("MP4", ".mp4"), ("FLV", ".flv")])

    # This is looping for the number of videos in the filepath (range(len(filepath))),
    # indexing the filepath (enumerate(filepath))
    for i in range(len(filepath)): # Indexing through filepath list
        displayFiles1.insert('', 'end', text= filepath[i], values=(filepath[i]))

    # Gets the name of the video and appends it to a list, also append @'s to
    # timeZeroList and embryoSizeList as default values for initialConditions function
    videoPaths = list(filepath)
    for i in videoPaths:
        name = i.split("/")
        vidName = name[len(name)-1]
        videoTitleList.append(vidName)
        timeZeroList.append("@")
        embryoSizeList.append("@")

## Define event: Removes all of the videos in the file list
def RemoveAllVideos():
    for child in displayFiles1.get_children():
        displayFiles1.delete(child)
    # Clears lists used in initialConditions
    videoTitleList.clear()
    embryoSizeList.clear()
    timeZeroList.clear()

## Define event: Remove selected file
def removeFile():
    try:
        #gets filepath for selected video and splices it into a list by '/'
        item=displayFiles1.item(displayFiles1.focus())
        videoPath=item['text'].split('/')

        selected_item1 = displayFiles1.selection()[0] # get selected item
        displayFiles1.delete(selected_item1)

        #get video name at the end of the list and removes it from videoTitleList
        videoPathName=videoPath[len(videoPath)-1]
        vidIndex=videoTitleList.index(videoPathName)
        videoTitleList.remove(videoPathName)
        #deletes the initialConditions value for the selected video at its respective i
        del embryoSizeList[vidIndex]
        del timeZeroList[vidIndex]

    except IndexError:
        messagebox.showerror("Error", "No selected file") # error message

## Define event: Remove the directories created for images and segmentations
def removeTemporaryDirs(org_folders, org2_folders, org3_folders, csvfile2):
    for i in org_folders:
        rmtree(i)

    for i in org2_folders:
        rmtree(i)

    for i in org3_folders:
        rmtree(i)

    os.remove(csvfile2)

```

Figure 8: Old upload/remove functions

```

122
123 def _update_conditions(self, event=None):
124     """
125     Update the Embryo Size and Percent Threshold
126     displayed when selecting a video
127     """
128     index = self._curselection()
129     if index is None or self.video_listbox.size() == 0:
130         self.embryo_size_value.configure(text="")
131         self.percent_threshold_value.configure(text="")
132         return
133
134     video_path = self.video_listbox.get(index)
135
136     if video_path not in self.initial_conditions:
137         self.embryo_size_value.configure(text="")
138         self.percent_threshold_value.configure(text="")
139         return
140
141     initial_conditions = self.initial_conditions[video_path]
142     self.embryo_size_value.configure(text=initial_conditions["embryo_size"])
143     self.percent_threshold_value.configure(text=initial_conditions["percent_threshold"])
144
145
146 def _run_analysis(self):
147     """
148     Creates an instance of the VideoAnalysis class.
149
150     This will run segmentation and display the results window. Do not store instance in a variable, it gets
151     destroyed when finished so lists and other data don't have to be manually reset. When returning to the
152     main menu screen to upload new videos, all video data is destroyed with that instance.
153     """
154     # If no videos are uploaded, <Analyze> button does nothing
155     if self.video_listbox.size() == 0:
156         CtkMessageBox(title="Error", message="No files uploaded.")
157         return
158
159     # Set progressbar for analysis
160     self.conditions.pack_forget()
161     self.buttons.pack_forget()
162
163     self.progress.pack(pady=5)
164     self.progressbar.pack()
165     self.progressbar.set(0)
166     self.percentage.pack(pady=5)
167
168     # Call two updates here. Needed for progressbar to appear right away
169     self.app.update()
170     self.app.update()
171
172     # Run Analysis and return the results
173     results = VideoAnalysis(self.app).run()
174
175     # AnalyzedResults(self.app, results)
176
177     self.app.bell()
178     self.app.main_menu.pack_forget()
179     results_window = AnalyzedResults(self.app, results)
180     results_window.pack(fill="both", expand=True)

```

Figure 9: Added methods

As for `initial_conditions.py` which defines the `InitialConditions` class, a small pop-up window that allows the user to set the initial embryo size and percent threshold for a specific video. As shown in Figure 10, it inherits `ctk.CTkToplevel`, making it a secondary window that floats above

the main application window. It centers itself relative to the main window rather than the screen, so it always appears near the application regardless of where the window is positioned.

Previously, as shown in Figure 11, the application required the user to navigate through two separate popup windows to set initial conditions: the first popup contained a listbox for selecting a video, and clicking “Select” would open a second popup for entering the values. This was redundant, since the main menu already had a listbox for selecting videos. To address this issue, we removed that additional pop-up window that asks to select a video entirely, since we are able to select a video from the listbox as shown in Figure 10. The InitialConditions popup now opens directly for whichever video is selected in the main menu listbox and displays that video’s filename as a header so the user always knows which video they are editing. Below the header are the same two labeled input fields for the initial embryo size and percent threshold, followed by a done button.

```

1  import customtkinter as ctk
2  from modules.CTkMessagebox.ctkmessagebox import CTkMessagebox
3
4
5  class InitialConditions(ctk.CTkToplevel):
6      """Popup window to set the initial embryo size and percent threshold."""
7
8      def __init__(self, app, video_path):
9          super().__init__(app)
10         self.app = app
11         self.video = video_path
12
13
14         # Setup window
15         self.title("Edit Initial Conditions")
16         self.after(200, lambda: self.iconbitmap(self.app.icon))
17         self.resizable(False, False)
18         self.configure(fg_color="■ #f1f1f1")
19
20         # Center window
21         width, height = 500, 300
22         app_width, app_height = self.app.winfo_width(), self.app.winfo_height()
23         app_x, app_y = self.app.winfo_x(), self.app.winfo_y()
24         x, y = app_x + (app_width - width) // 2, app_y + (app_height - height) // 2
25         self.geometry(f"{width}x{height}+{x}+{y}")
26
27         self.grab_set()
28         self.focus()
29
30
31         # Header
32         video_name = self.video.split("/")[-1]
33         ctk.CTkLabel(self, text=video_name, font=("Poppins", 18, "bold")).pack(fill="x", pady=(50, 0))
34
35
36         # Initial Embryo Size
37         frame = ctk.CTkFrame(self, fg_color="transparent")
38         frame.pack(fill="x", padx=10, pady=(30, 10))
39         ctk.CTkLabel(frame, text="Initial Embryo Size ( $\mu\text{m}^2$ ):", font=("Poppins", 12)).pack(side="left", padx=(80, 0))
40         self.embryo_size_input = ctk.CTkEntry(frame, width=100)
41         self.embryo_size_input.pack(side="right", padx=(0, 80))
42
43         # Percent Threshold
44         frame = ctk.CTkFrame(self, fg_color="transparent")
45         frame.pack(fill="x", padx=10)
46         ctk.CTkLabel(frame, text="Percent Threshold (%):", font=("Poppins", 12)).pack(side="left", padx=(80, 0))
47         self.percent_threshold_input = ctk.CTkEntry(frame, width=100)
48         self.percent_threshold_input.pack(side="right", padx=(0, 80))
49
50
51         # Done button
52         ctk.CTkButton(self, text="Done", command=self._set_conditions).pack(side="bottom", pady=30)
53
54
55         self.protocol("WM_DELETE_WINDOW", self.destroy)

```

Figure 10: Optimized initial conditions

```

def initialConditions():
    ## When user clicks on a video and clicks "Select", it will bring up a window that allows input for initial conditions
    def select():
        selectedVid = videoListbox.get(ANCHOR)
        if selectedVid:
            vidIndex = videoTitleList.index(selectedVid)
            selectedLabel.config(text=selectedVid)
            popupWindow = tk.Toplevel(root)
            popupWindow.title("Configure Data")
            popupWindow.geometry("450x200")
            popupWindow.iconbitmap(icon)
            # popupWindow.overridedirect(True)
            popupWindow.configure(bg="#282a36")
            popupWindow.resizable(width=False, height=False)

            ## After user enters initial conditions and clicks "Done" this function will check and append
            # data to embryoSizeList and timeZeroList at its respective index relative to the video title
            def getInputValues():
                def check_float(potential_float):
                    try:
                        float(potential_float)
                        return True
                    except ValueError:
                        return False

                if check_float(timeZeroInput.get()):
                    timeZeroList[vidIndex]=timeZeroInput.get()

                if check_float(embryoSizeInput.get()):
                    embryoSizeList[vidIndex]=embryoSizeInput.get()

            # Display Initial Embryo Size label and entry box
            initialEmbryoSizeText = ttk.Label(popupWindow, text = "Initial Embryo Size ( $\mu\text{m}^2$ ):", font = ("Helvetica", 11))
            initialEmbryoSizeText.place(relx=0.1, rely=0.2)

            embryoSizeInput = tk.Entry(popupWindow, width=15)
            embryoSizeInput.place(relx=0.5, rely=0.2)

            # Display Time Zero Label and entry box
            timeZeroText = ttk.Label(popupWindow, text = "Percent Threshold (%)ate", font = ("Helvetica", 11))
            timeZeroText.place(relx=0.1, rely=0.4)

            timeZeroInput = tk.Entry(popupWindow, width=15)
            timeZeroInput.place(relx=0.5, rely=0.4)

```

Figure 11: Old initial conditions function

When Done is clicked, the `_set_conditions` method shown in Figure 12 reads both input fields and attempts to convert them to floats. If either value is non-numeric, a `CTkMessagebox` error dialog is displayed and the popup remains open so the user can correct the input. If both values are valid, they are written into the `initial_conditions` dictionary on `MainMenu`, and `_update_conditions` in Figure 9 is called to immediately refresh the inline display on the main menu before the popup closes.


```

58     def _set_conditions(self):
59         """Set the Embryo Size and Percent Threshold values to the user's input."""
60         embryo_size = self.embryo_size_input.get()
61         percent_threshold = self.percent_threshold_input.get()
62
63         try:
64             if embryo_size:
65                 float(embryo_size)
66                 self.app.main_menu.initial_conditions[self.video]["embryo_size"] = embryo_size
67             if percent_threshold:
68                 float(percent_threshold)
69                 self.app.main_menu.initial_conditions[self.video]["percent_threshold"] = percent_threshold
70
71             self.app.main_menu._update_conditions()
72             self.destroy()
73
74         except ValueError:
75             tkinter.messagebox(title="Error", message="Inputs must be numeric")
76         return

```

Figure 12: Added displayed initial conditions method

In Figure 13, `get_timestamp_crop_coords` calculates the pixel coordinates for cropping the timestamp region out of a video frame based on the image resolution and the number of digits in the timestamp. This function is placed at the top, outside of the `VideoAnalysis` class because it is not using anything in the class and is only being used in the `extract frames` function. If a function is using `self`, then it goes as a method since it needs to access attributes of the class, otherwise it does not need to be a method of the class. This is an important distinction: if a function uses `self`, it belongs as a method of the class because it needs access to the object's state. If it does not, placing it outside the class keeps the class focused on its own responsibilities and makes the function independently reusable.

```

1  import os
2  import csv
3  import numpy as np
4  import cv2
5  from PIL import Image, ImageEnhance
6  from pandas import DataFrame
7  from shutil import rmtree
8
9  import pytesseract
10
11
12  PIXEL_TO_REAL = 0.58 * 0.58
13
14
15  def get_timestamp_crop_coords(image_size, num_digits):
16      """
17      Calculate the coordinates for cropping the timestamp
18      region based on the image size and number of digits.
19      """
20      if image_size == (500, 500):
21          if num_digits == 3:
22              return (455, 475, 483, 489) # Smaller box for 3 digits
23          elif num_digits == 4:
24              return (449, 475, 483, 489) # Larger box for 4 digits
25      elif image_size == (800, 800):
26          if num_digits == 3:
27              return (736, 769, 775, 787) # Smaller box for 3 digits
28          elif num_digits == 4:
29              return (731, 769, 785, 795) # Larger box for 4 digits
30      else:
31          raise ValueError("Unsupported image size")

```

Figure 13: Coordinates of timestamp function

In `video_analysis.py`, it defines the `VideoAnalysis` class, which contains the entire analysis pipeline. It has no UI responsibilities — it processes videos, runs segmentation, calculates measurements, and returns results. In its constructor it reads all uploaded videos and their conditions from `app.main_menu.initial_conditions`, building a list of video dictionaries. Each dictionary stores everything related to one video: file path, name, input and output directory paths, frame count, timestamps, size measurements, growth rate, and data frames as shown in Figure 14. It also holds references to the progress bar widgets on `MainMenu` so it can update them throughout the analysis.

```

34 class VideoAnalysis:
35     """
36     Handles video analysis. Processes videos, runs
37     segmentation, calculates embryo size and growth rates.
38     """
39     def __init__(self, app):
40         self.app = app
41         self.model = app.model
42
43
44         self.videos = []
45         for path, conditions in app.main_menu.initial_conditions.items():
46             data = {
47                 "path": path, # Path to video file
48                 "name": None, # Video name with extension removed
49                 "embryo_size": conditions["embryo_size"],
50                 "percent_threshold": conditions["percent_threshold"],
51
52                 "images_dir": None, # Input images dir for segmentation
53                 "masks_dir": None, # Output masks dir for segmentation
54                 "overlays_dir": None, # Output overlays dir for segmentation
55
56                 "frame_count": 0,
57                 "raw_final_time": None,
58
59                 "initial_size": None,
60                 "initial_time": None,
61                 "final_size": None,
62                 "final_time": None,
63                 "growth_rate": None,
64
65                 "size_data": None, # DataFrame
66                 "growth_data": None, # DataFrame
67             }
68             self.videos.append(data)
69
70         self.video_data = [] # [name, final_size, growth_rate]
71         self.size_ranking_data = [] # Data for Size Ranking Table (Treeview)
72         self.growth_ranking_data = [] # Data for Growth Rate Ranking Table (Treeview)
73         self.rankings_csv = os.path.join(self.app.base_path, "EmbryoGrowthRateRanking.csv")
74
75         self.time_array = []
76         self.frame_count = 0
77
78         # Track video being analyzed
79         self.current_video_index = 0
80         self.current_video = None # Current video dict from self.videos
81
82         self.failed = [] # Track videos that failed to process
83
84         # Setup progressbar for analysis
85         self.progressbar = app.main_menu.progressbar
86         self.progress = app.main_menu.progress
87         self.percentage = app.main_menu.percentage
88         self.num_videos = len(self.videos)
89         self.slice_size = 1.0 / self.num_videos

```

Figure 14: video_analysis.py attributes

In Figure 15, the run() method loops through every video and calls the six internal methods in sequence for each one: _create_output_dirs, _extract_frames, _run_segmentation, _count_pixels, _create_ranking_data, and _write_rankings_csv which is in Figures 17 to 20 and 22 to 23.

```
92     def run(self):
93         for video_index, video in enumerate(self.videos):
94             self.current_video_index = video_index
95             self.current_video = video
96
97             self._create_output_dirs()
98             self._extract_frames()
99             self._run_segmentation()
100            self._count_pixels()
101
102            # If no embryo was detected, track to be removed
103            if self.current_video["growth_rate"] is None:
104                print(f"Skipping {video['name']}: no embryo detected.")
105                self.failed.append(video)
106
107            for video in self.failed:
108                self.videos.remove(video)
109
110            # Create Results
111            self._write_rankings_csv()
112            self._create_ranking_data()
113
114            results = {
115                "images_dirs": [vid["images_dir"] for vid in self.videos],
116                "masks_dirs": [vid["masks_dir"] for vid in self.videos],
117                "overlays_dirs": [vid["overlays_dir"] for vid in self.videos],
118
119                "video_data": self.video_data,
120                "file_names": [vid["name"] for vid in self.videos],
121
122                "size_data": [vid["size_data"] for vid in self.videos],
123                "growth_data": [vid["growth_data"] for vid in self.videos],
124
125                "raw_final_times": [vid["raw_final_time"] for vid in self.videos],
126                "final_sizes": [vid["final_size"] for vid in self.videos],
127
128                "size_ranking_data": self.size_ranking_data,
129                "growth_ranking_data": self.growth_ranking_data,
130
131                "rankings_csv": self.rankings_csv
132            }
133
134            return results
```

Figure 15: video_analysis methods

In Figure 16, the _create_output_dirs function derives the video's name by stripping the file extension from its path and then creates three directories: one for the extracted frames, one for the segmentation masks, and one for the overlay images. In the previous semester's codebase, shown in Figure 17, these directories were created immediately when videos were uploaded rather than when analysis began, which is why a separate function was needed to remove them afterward. To circumvent this, we created these directories only when the user decides to analyze their video(s), ensuring the user has finalized their choice that is why it is the first function being

used in the run function. If any of these directories already exist from a previous run, they are deleted first to ensure a clean slate.

```
def _create_output_dirs(self):
    """Create the output folders for segmentation."""
    name = os.path.splitext(os.path.basename(self.current_video["path"]))[0]
    self.current_video["name"] = name

    self.current_video["images_dir"] = os.path.join(self.app.base_path, f"{name}Images")
    self.current_video["masks_dir"] = os.path.join(self.app.base_path, f"{name}Seg")
    self.current_video["overlays_dir"] = os.path.join(self.app.base_path, f"{name}Seg1")

    for path in (self.current_video["images_dir"], self.current_video["masks_dir"], self.current_video["overlays_dir"]):
        if os.path.exists(path):
            rmtree(path)
        os.mkdir(path)
```

Figure 16: Implemented and adjusted code

```
# Creates the images and segmentation folders to store images
temp_path = homedir + '/%s' % (filename) + 'Images/' # folder for the OG images being fed in
os.mkdir(temp_path)
temp_path2 = homedir + '/%s' % (filename) + 'Seg/' # folder for just the mask output
os.mkdir(temp_path2)
temp_path3 = homedir + '/%s' % (filename) + 'Seg1/' # folder for the overlay output
os.mkdir(temp_path3)

#debug: numbers
# temp_path4 = homedir + '/%s' % (filename) + 'Numbers/'
# os.mkdir(temp_path4)

# Store information about video into the list
org_folders += [temp_path]
org2_folders += [temp_path2]
org3_folders += [temp_path3]
filelist += [filename]
listoftemp_path3.append(temp_path3)
```

Figure 17: Old semester's folder storage

The three functions in Figures 18 to 20 are the main process of how the video is being analyzed, where we separated each part of the process into methods. Using the `_extract_frames` function to open the video file using OpenCV and reads it frame by frame. For each frame it crops a small region where the timestamp is embedded in the image, converts that region to black and white, and passes it to Tesseract OCR to extract the time value. It uses `get_timestamp_crop_coords` in Figure 13 to calculate the exact crop coordinates based on both the image resolution and number of digits in the timestamp, since the timestamp region shifts slightly when the value transitions from three digits to four. Each extracted frame is saved as a numbered PNG file in the images directory, and the progress bar is updated as frames are extracted.

```

def _extract_frames(self):
    self.time_array.clear()
    raw_time_array = []
    self.frame_count = 1

    # Reads the video from the list
    video = cv2.VideoCapture(self.current_video["path"])
    success, image = video.read()

    num_digits = 3
    factor = 0.3047
    max_factor = 5.0
    increase_step = 0.5
    prev_extracted_time = None
    prev_M = None
    max_retries = 5
    estimated_frame_count = int(video.get(cv2.CAP_PROP_FRAME_COUNT))

    while success:
        # Changes the image to gray scale
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

        # Get the size of the image
        image_size = (image.shape[1], image.shape[0])
        found = False
        retries = 0

        while not found and retries < max_retries:
            # Calculate coordinates for timestamp region
            top_left_x, top_left_y, bottom_right_x, bottom_right_y = get_timestamp_crop_coords(image_size, num_digits)

            # Get the space between third and second digit using calculated coordinates
            digitSpace = image[top_left_y:bottom_right_y, top_left_x:bottom_right_x]

```

Figure 18: video_analysis.py _extract_frames

The `_run_segmentation` function shown in Figure 19 is intentionally short because its work is delegated to the ResUNet-50 model's `predict_multiple` method from the `keras_segmentation` library. It passes the images directory as input and the masks and overlays directories as outputs. `Predict_multiple` is a built-in method from `keras_segmentation` library specifically from the pretrained module. The model processes every image in the input directory and writes a segmentation mask and a color overlay image for each one. The progress bar label is updated to indicate that segmentation is running.

```

319
320     def _run_segmentation(self):
321         self.progress.configure(text=f"{self.current_video['name']}: Running Segmentation")
322         self.app.update()
323
324         # Run segmentation
325         self.model.predict_multiple(
326             inp_dir=self.current_video["images_dir"],
327             out_dir_mask=self.current_video["masks_dir"],
328             out_dir_fused=self.current_video["overlays_dir"]
329         )

```

Figure 19: video_analysis.py _run_segmentation

This segment of the code allows the application to first count the pixels of the embryo in just the first and last frame of the video of an embryo, essentially acting as a sanity check for the application.

Next, `_count_pixels` loops through every mask image and counts the number of pixels belonging to the embryo class, identified by a specific RGB color value. It converts the pixel count to a real-world area in μm^2 using a calibration constant. If the user provided an initial embryo size, a scaling factor is calculated from the first frame's pixel count to calibrate the subsequent measurements. A percent threshold is applied to filter out frames where the embryo has not yet reached its starting size. The resulting size values and their corresponding timestamps are stored in DataFrame. Linear regression is then applied to the last ten hours of data or the full dataset if the video is shorter than ten hours to calculate the average growth rate. The progress bar updates continuously throughout this step.

```

332 def _count_pixels(self):
333     pixelArray = []
334     i, starting_counting_size, temp_frame_number = 1, 0, []
335
336     # Loop through each video and count the amount of pixels from segmentation and store them
337     while i <= self.current_video["frame_count"]:
338         # Get the pixel count
339         img = Image.open(os.path.join(self.current_video["masks_dir"], f"frame{i}.png"))
340         img_array = np.array(img)
341         embryo_pixels = img_array == [10, 34, 46]
342         matching_pixels = np.all(embryo_pixels, axis=2)
343         pixel_count = int(np.sum(matching_pixels))
344
345         if i == 1:
346             # Get the scaling factor from the initial input size
347             if float(self.current_video["embryo_size"]) == 0 or pixel_count == 0:
348                 scaling_factor = 1
349             else:
350                 scaling_factor = float(self.current_video["embryo_size"]) / (pixel_count * PIXEL_TO_REAL)
351
352             # Gets the value for the threshold
353             multiplication_threshold = (float(self.current_video["percent_threshold"]) + 100) / 100
354             starting_counting_size = pixel_count * PIXEL_TO_REAL * scaling_factor * multiplication_threshold
355
356             # Checks to see if the size passes the threshold
357             if (pixel_count * PIXEL_TO_REAL * scaling_factor) >= starting_counting_size:
358                 temp_frame_number += [i]
359
360             # Store the pixel size to an array
361             pixelArray.append(pixel_count * PIXEL_TO_REAL * scaling_factor)
362             i += 1
363
364             pixel_progress = self.slice_size * self.current_video_index + self.slice_size * 0.6 + self.slice_size * 0.4 * (i / self.current_video["frame_count"])
365             self.progressbar.set(pixel_progress)
366             self.percentage.configure(text=f"{int(pixel_progress * 100)}% Completed")
367             self.app.update()
368
369
370     # Handle error if no embryo is detected in video
371     if not temp_frame_number:
372         return # Move on to next video
373
374     tempdeletenumber = temp_frame_number[0] - 1
375     self.time_array = self.time_array[tempdeletenumber:]
376     pixelArray = pixelArray[tempdeletenumber:]

```

Figure 20: video_analysis.py `_count_pixels`

Figures 21 and 22 below compare and contrast the updates to the algorithm used to rank the size and growth of the embryos analyzed. Figure 21 shows the tedious for loop that was previously used to loop through each video, involving the need for temporary arrays which take up cache storage on the user's device as well as reopening the CSV file. The `_create_ranking_data` method

allows the data extracted from the neural network to instead be read directly from the CSV file that is generated, allowing also for decreased computation time.

```
# Gets the data to display in the tables
for i in data[1:]:
    tempRankingData = []
    tempRank = float((i[6]))
    tempVideo = i[0]
    tempAverageGrowth = i[5]
    tempRankingData.append(tempVideo)
    tempRankingData.append(tempRank)
    tempRankingData.append(tempAverageGrowth)
    RankingDataGR.append(tempRankingData)

#debug statement: #print(RankingDataGR)

## Current Video Data
CurrentVideoData = []

# Reopen the CSV file
with open(csvfile2, newline='') as f:
    reader = csv.reader(f)
    data = list(reader)

# Current video amount
CurrentTotalVideoAmount = 0

# Gets the data to display in the tables
for i in data[1:]:
    tempVideoData = []
    tempFinalSize = float((i[3]))
    tempAverageGR = float((i[5]))
    tempVideoData.append(i[0])
    tempVideoData.append(tempFinalSize)
    tempVideoData.append(tempAverageGR)
    CurrentVideoData.append(tempVideoData)
    CurrentTotalVideoAmount += 1

#debug statement: #print(CurrentVideoData)

# Sorting Ranking Data
RankingDataSize.sort(key=lambda e: e[1])
RankingDataGR.sort(key=lambda e: e[1])
```

Figure 21: Old ranking data code


```

423
424     def _create_ranking_data(self):
425         """Read the rankings CSV file to create ranking data."""
426         with open(self.rankings_csv, newline='') as f:
427             reader = csv.reader(f)
428             data = list(reader)
429
430         for item in data[1:]:
431             video = item[0]
432
433             size_rank = float((item[4]))
434             final_size = item[3]
435             self.size_ranking_data.append([video, size_rank, final_size])
436
437             growth_rank = float((item[6]))
438             avg_growth = item[5]
439             self.growth_ranking_data.append([video, growth_rank, avg_growth])
440
441             final_size = float(final_size)
442             avg_growth = float(avg_growth)
443             self.video_data.append([video, final_size, avg_growth])
444
445         # Sorting Ranking Data
446         self.size_ranking_data.sort(key=lambda e: e[1])
447         self.growth_ranking_data.sort(key=lambda e: e[1])
448

```

Figure 22: video_analysis_create_ranking_data

After the creation of the ranking data, the `_write_rankings_csv` method is then used to neatly create a new CSV file to rank and store the embryo size and growth data. The method first creates the CSV file then compares the size and growth of each embryo against one another, writing them into the CSV sorted from best to worst in regards to both final size and growth rate.

```

449
450 def _write_rankings_csv(self):
451     # Write results to CSV
452     field_names = ["Videos", "Time (h)", "Initial Size ( $\mu\text{m}^2$ )", "Final Size ( $\mu\text{m}^2$ )",
453                   "Final Size Rank", "Average Growth Rate ( $\mu\text{m}^2/\text{h}$ )", "Avg Growth Rate Rank"]
454
455     # Create Rankings CSV file and write data to it
456     with open(self.rankings_csv, "w", newline="") as file:
457         writer = csv.DictWriter(file, fieldnames=field_names)
458         writer.writeheader()
459
460         for video in self.videos:
461             writer.writerow({
462                 "Videos": video["name"],
463                 "Time (h)": video["final_time"],
464                 "Initial Size ( $\mu\text{m}^2$ )": video["initial_size"],
465                 "Final Size ( $\mu\text{m}^2$ )": video["final_size"],
466                 "Final Size Rank": 1,
467                 "Average Growth Rate ( $\mu\text{m}^2/\text{h}$ )": video["growth_rate"],
468                 "Avg Growth Rate Rank": 1,
469             })
470
471     # Rerank videos by final size and growth rate
472     if len(self.videos) > 1:
473         with open(self.rankings_csv, newline="") as file:
474             reader = csv.reader(file)
475             data = list(reader)
476
477             size = [] # Size Ranking
478             growth = [] # Growth Ranking
479
480             # Get size and growth data
481             for item in data[1:]:
482                 size.append(float(item[3]))
483                 growth.append(int(float(item[5])))
484
485
486             size_ranks = [0] * len(size)
487             for i in range(len(size)):
488                 (r, s) = (1, 1)
489                 for j in range(len(size)):
490                     if j != i and size[j] > size[i]:
491                         r += 1
492                     if j != i and size[j] == size[i]:
493                         s += 1
494
495                 # Use formula to obtain rank
496                 size_ranks[i] = int(r + (s - 1) / 2)
497
498             # Replace preset ranking with new ranking
499             for i, item in enumerate(data[1:]):
500                 item[4] = str(size_ranks[i])
501
502
503             growth_ranks = [0] * len(growth)
504             for i in range(len(growth)):
505                 (r, s) = (1, 1)
506                 for j in range(len(growth)):
507                     if j != i and growth[j] > growth[i]:
508                         r += 1
509                     if j != i and growth[j] == growth[i]:

```

Figure 23: video_analysis_write_ranking_csv

After analysis of the videos that are uploaded to the application, the main.py script calls upon the AnalyzedResults class to generate the window where the user may view the results of the embryo analysis. The attributes of the class are first initialized, carrying over the directories to images, masks, and CSV files which were used to store data in the VideoAnalysis class.

```
1 import os
2 import cv2
3 from PIL import Image
4 from natsort import natsorted
5
6 import customtkinter as ctk
7 from CTkTable import CTkTable
8
9 from shutil import copy, copytree, rmtree
10
11 import matplotlib.pyplot as plt
12 from matplotlib.backends.backend_tkagg import (FigureCanvasTkAgg, NavigationToolbar2Tk)
13
14
15 def remove_tree(folders):
16     for i in folders:
17         if os.path.exists(i):
18             rmtree(i)
19
20
21 class AnalyzedResults(ctk.CTkFrame):
22     """Displays the results window."""
23
24     def __init__(self, app, results):
25         super().__init__(app)
26         self.app = app
27
28         self.images_dirs = results["images_dirs"]
29         self.masks_dirs = results["masks_dirs"]
30         self.overlays_dirs = results["overlays_dirs"]
31         self.rankings_csv = results["rankings_csv"]
32
33         self.video_data = results["video_data"]
34         self.file_names = results["file_names"]
35
36         self.size_data = results["size_data"]
37         self.growth_data = results["growth_data"]
38
39         self.size_ranking_data = results["size_ranking_data"]
40         self.growth_ranking_data = results["growth_ranking_data"]
41
42         self.raw_final_times = results["raw_final_times"]
43         self.final_sizes = results["final_sizes"]
44
45         self.current_video_index = 0
46
47         self.plot_figure = None # plot_data figure
48         self.plot_canvas = None # plot_data canvas
49
50
51         self.configure(fg_color="transparent")
52
53
54         # Top navigation bar
55         nav = ctk.CTkFrame(self, fg_color="transparent")
56         nav.pack(fill="x", padx=10, pady=10)
57         ctk.CTkButton(nav, width=40, text="?", command=app.open_help).pack(side="right")
58
59
60         # Menu / Navigation buttons added into app.nav frame
61         options = ["Load New Videos", "Save Images & Labels", "Save CSV Results", "Save Current Plot", "Comparison Plot", "Exit"]
62         self.option_menu = ctk.CTkOptionMenu(nav, values=options, font=("Times New Roman", 14, "bold"), fg_color="#278bf5", command=self._selected_option)
63         self.option_menu.set("Menu")
64         self.option_menu.pack(side="left")
```

Figure 24: analyzed_results.py attributes

Many of the methods used here are identical to those at the top of the OpenNewWindow function that was previously used in the old version of the application due to the carryover of the menu button. The _selected_options method executes the appropriate function when selecting an option from the drop down menu. Meanwhile, the _load_new_videos method deletes the

temporary files and folders and allows the user to return to the main menu to load more videos, allowing to run the application with a clean slate.

```
229
230     def _selected_option(self, selection):
231         """
232         Executes the function when selecting an
233         option from the drop down menu.
234         """
235         self.option_menu.set("Menu")
236         if selection == "Load New Videos":
237             self._load_new_videos()
238         elif selection == "Save Images & Labels":
239             self._save_images()
240         elif selection == "Save CSV Results":
241             self._save_results()
242         elif selection == "Save Current Plot":
243             self._save_current_plot()
244         elif selection == "Comparison Plot":
245             self._gen_comparison_plot()
246         elif selection == "Exit":
247             self._on_closing()
248
249
250     def _load_new_videos(self):
251         """Delete temp files/folders and return to the main menu."""
252         self._remove_temp_dirs()
253         plt.close(self.plot_figure)
254         self.app.unbind("<Configure>")
255         self.app.protocol("WM_DELETE_WINDOW", self.app.destroy)
256         self.destroy()
257         self.app.create_main_menu()
258
```

Figure 25: analyzed_results.py methods

The getNetFrameCount function in Figure 26 was previously used in the old code to get the total number of frames that were in a single video to be used in the generation of the graphs and visuals in the user interface. In the modular application, this was instead changed to the `_get_frame_count` method shown in Figure 27, which simply returns the number of images in the overlay folder rather than having to sort and count through an array.

```

# def getNetFrameCount(CurrentTotalVideoAmount, CurrentVideoData, value):
#     CurrentVideoNumber = GetCurrentNumber()
#     # update video name
#     # check number of files in current video path
#     totalcurrentCount = 0
#     for f in os.scandir(org3_folders[currentpathNum]):
#         if f.is_file():
#             totalcurrentCount += 1
#     listoftemppath3[currentpathNum])
#     # check number of files in current video path
#     totalcurrentCount = 0
#     for f in os.scandir(listoftemppath3[CurrentVideoNumber]):
#         if f.is_file():
#             totalcurrentCount += 1

```

Figure 26: Commented out code

Meanwhile, the `_on_resize` method that is also mentioned in Figure 27 is used to aid in resizing the image as a user resizes their window. In the previous code, this caused issues with the user interface as the frame of the embryo video that was on display was fixed.

```

def _get_frame_count(self):
    """Return the number of images in the overlay folder"""
    overlay_dir = self.overlays_dirs[self.current_video_index]
    return sum(1 for f in os.scandir(overlay_dir) if f.is_file())

def _on_resize(self, event=None):
    """Scales embryo image based on current window size."""
    if event.widget == self.app:
        self._on_scale_change(self.slider.get())

```

Figure 27: analyzed_results.py added/adjusted methods

As shown throughout the figures above, we standardized all naming conventions according to Python's official style guide, PEP 8. This includes the use of Python docstrings, which are documentation strings that describe the purpose and functionality of a module, class, method, or function [1]. Every method we created includes a docstring explaining what it does. For variables, functions, methods, and modules, we use snake case (e.g., snake_case). For private methods within a class, we prefix the name with an underscore (e.g., `_get_frame_count`) to signal that the method is an internal implementation detail. For class names, we use camel case (e.g. CamelCase). For constants, we use uppercase snake case (e.g. UPPER_SNAKE_CASE).

All in all, the transition to a modular codebase significantly improves the maintainability and scalability of the application, where each module now operates independently. VideoAnalysis has no knowledge of the user interface (UI); AnalyzedResults has no knowledge of how segmentation works. This separation means that future teams can swap out the segmentation model, for example replacing ResUNet-50 with nnUNet inside video_analysis.py without touching any UI file, or redesign the results screen without affecting the analysis logic. All naming conventions were standardized following PEP 8, and every method includes a docstring describing its purpose, making the codebase significantly easier for new team members to navigate. These changes reduce debugging time, improve readability, and ensure that the application remains extensible as new features and models are integrated in future semesters.

User Interface:

Another goal this semester was to update all Tkinter components to CustomTkinter for a more modern and consistent user interface. For the main menu window, we replaced the Tkinter treeview used as a file list with a CTkListbox, which includes a scrolling bar. We also added an inline display for the initial conditions of each video through the method `_update_conditions` shown in Figure 9. Figure 28 displays how the user had to open a separate popup window containing a second listbox just to select the video before they could edit its conditions. We have eliminated the first popup window as shown in Figure 29, so now a user selects a video from the listbox, its current embryo size and percent threshold are displayed immediately below without requiring any additional interaction. The progress bar is also fully functional with multiple videos, displaying the current progression, percentage completion, and which process the application is currently running as shown in Figure 30.

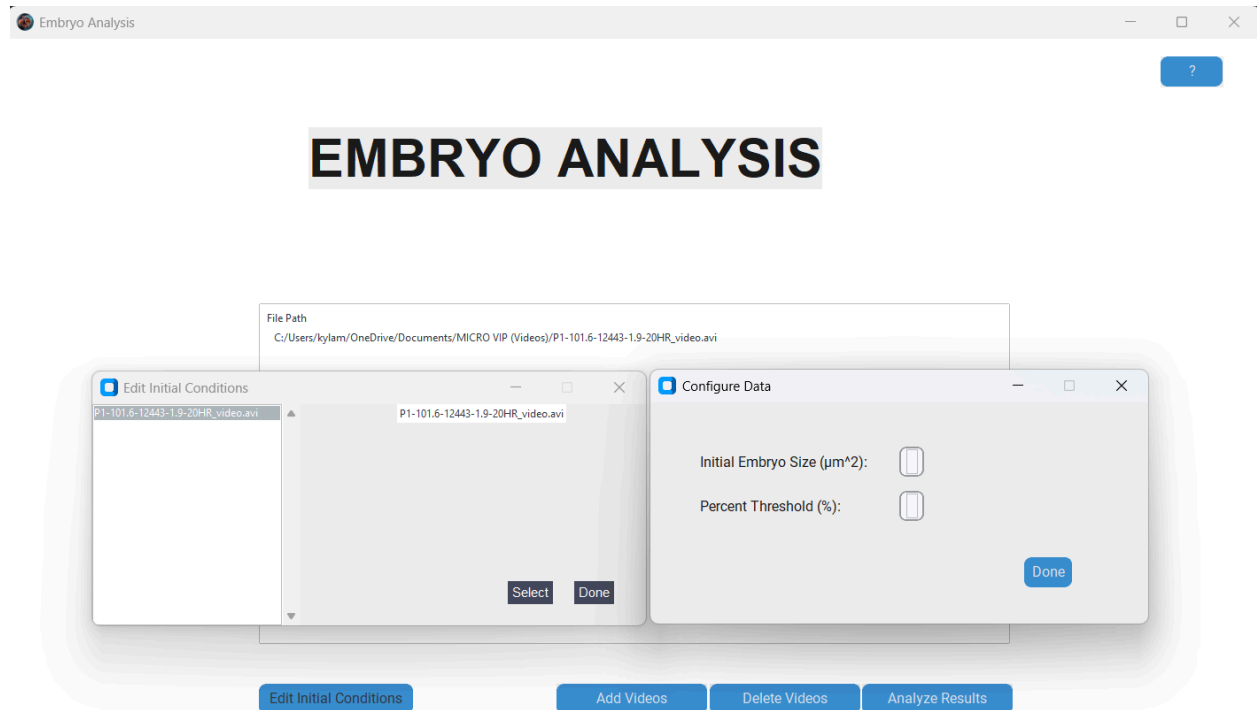


Figure 28: Old Main UI with Initial Conditions

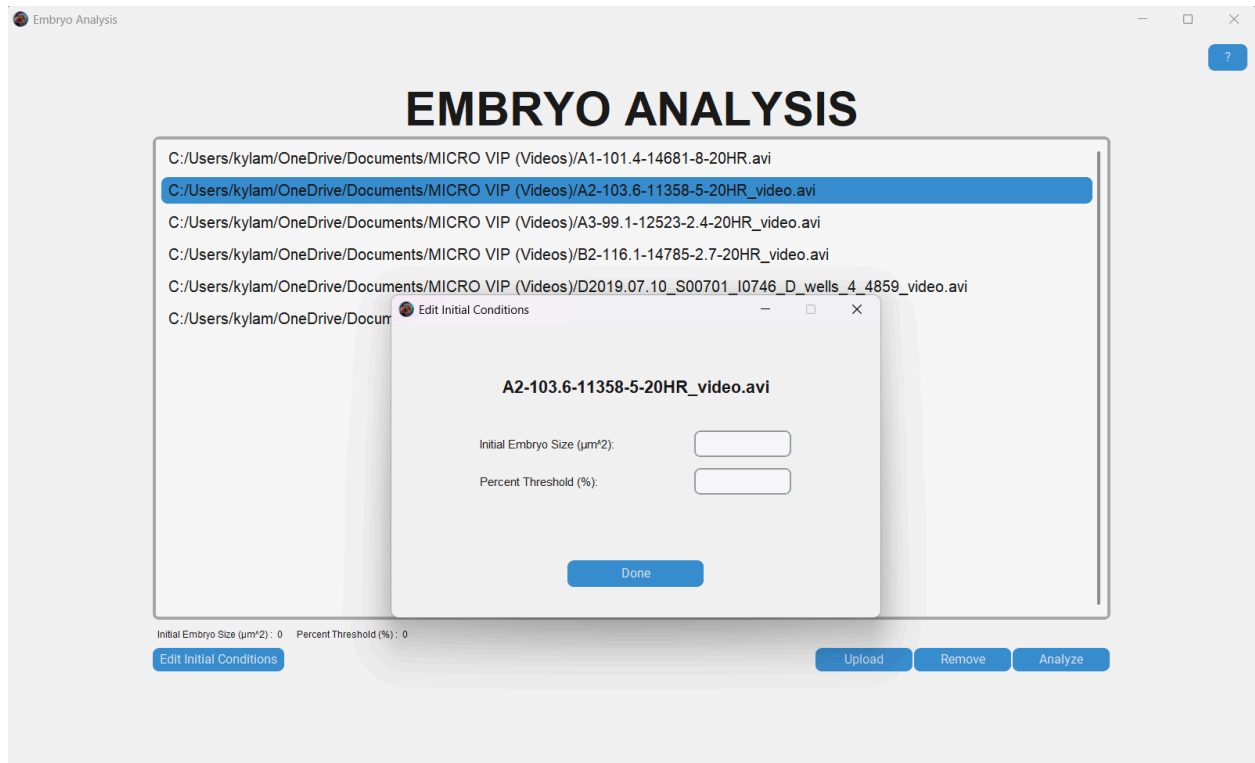


Figure 29: Current Main UI with Initial Conditions

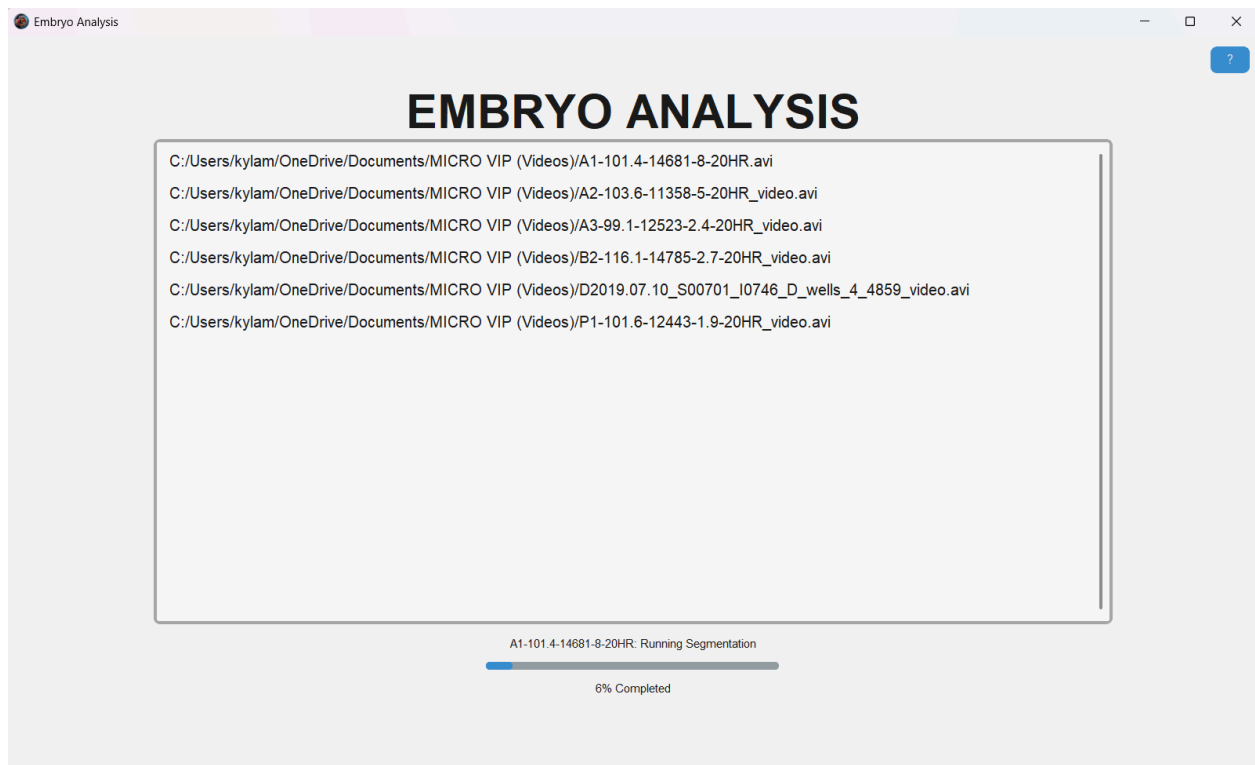


Figure 30: Current Main UI during Analysis

For the analyzed results window, we have reorganized the layout from Figure 31 to Figure 33. We created a navigation bar at the top which now contains the dropdown menu, navigation arrows, and the help button. The current video table was moved from below the image slider to directly under the navigation bar, making it immediately visible when the results screen opens. We added a CTkLabel for the current frame number above the slider, so the user always knows which frame they are viewing. All three data tables – current video table, size ranking table, and the average growth rate ranking table were replaced with CTkinter table, completing the full transition from Tkinter to CustomTkinter and providing consistent styling throughout the application. Now the current UI shows the all video’s current video data, graphs, and frames of their embryo video. Font sizes were increased across all components to improve readability, so that our users are able to clearly see the data and graphs.

Another feature we added to the analyzed results window is making the window resizable. The most significant structural change to the results window was replaced `.place()` with `.pack()` for all layout management. The previous version used `.place()` with hardcoded relative coordinates to position every widget, which caused them to disappear or overlap whenever the window was resized. This is why the analyzed results window in the previous version was locked and could not be resized by the user as shown in Figure 31. By switching to `.pack()`, all widgets adjust automatically when the window is resized as shown in Figure 32. Combined with the minimum window size enforced in `app.py`, users can resize the window freely while being prevented from shrinking it to the point where they are unable to see the content. Now the widgets in the frame or window (depending on the parent widget) will be aligned perfectly, unlike `.place()` where we need the specific coordinates as shown in Figure 33.

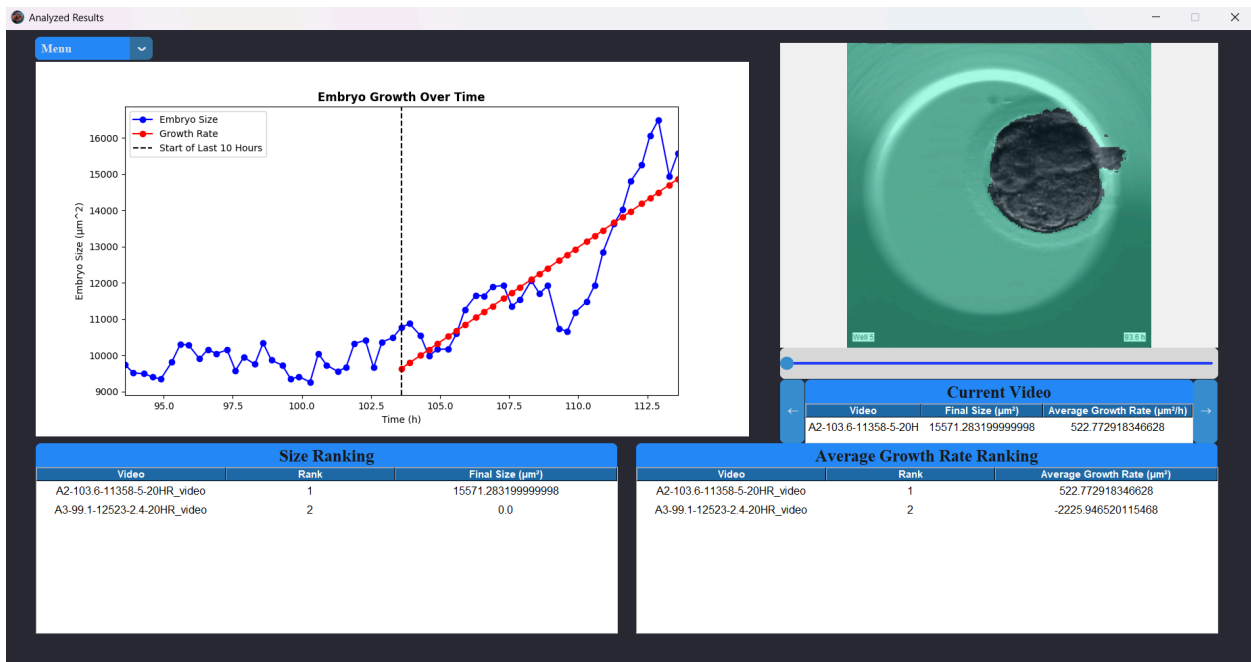


Figure 31: Old analyzed results UI

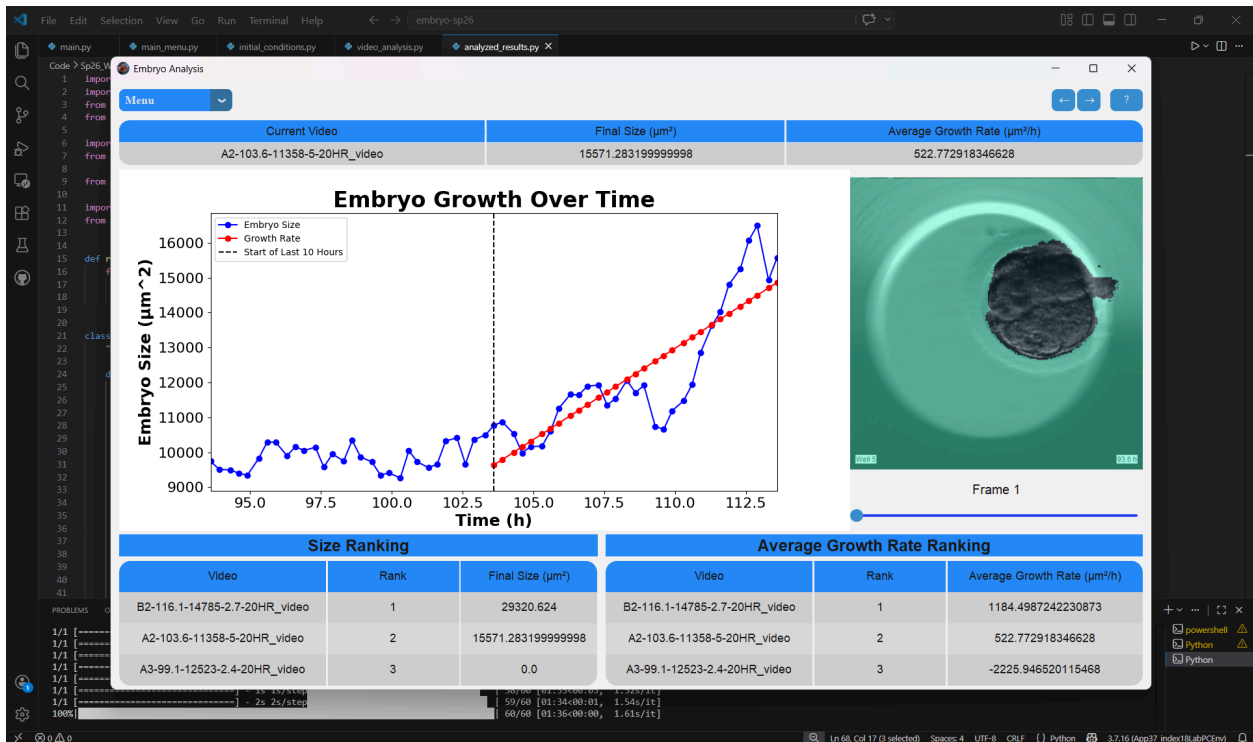


Figure 32: Resized analyzed results UI

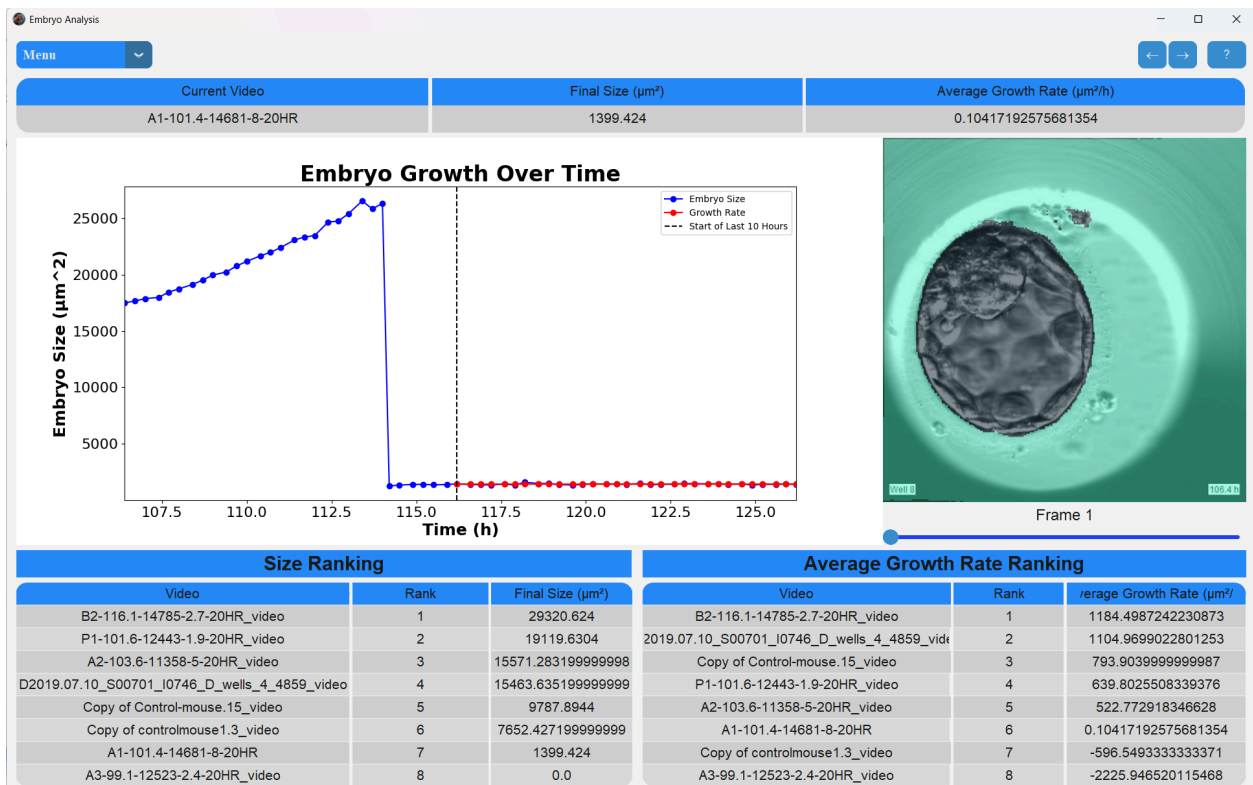


Figure 33: Finalized analyzed results UI

Embryo Application Deployment

Due to the many developments with the embryo application over the past two years, we decided to export the latest version of our work to a standalone application for the embryologist we are partnered with, Dr. Thomas Huang, and his clinic to use.

Building upon previous semesters' documentation, we chose to use the pyinstaller library [1] to export our code. There were, as always, some issues with the application exporting process as even simply following the documentation from a few semesters ago caused the application to freeze upon analyzing the videos. The two main road blocks in this regard were that Tensorflow which was used in the video analysis by the neural networks required somewhere to write its output to and that the application needed the baseweights of ResUNet-50 in order to also use the weights from our trained model.

Figure 34 below features the paths to the debug text file that is generated when the application is analyzing the embryo videos. Unless the user goes to the file directory and selects this debug.txt file, it is hidden from the user's view, ensuring that there is no confusion on the user's end.

```
19 # IMPORTANT: Uncomment when running PyInstaller
20 #sys.stdout = open(os.path.join(output_path, "debug.txt"), 'w', buffering=1)
21 #sys.stderr = open(os.path.join(output_path, "debug.txt"), 'a', buffering=1)
```

Figure 34: Paths to debug file

Figure 35 addresses the issue with the base weights for ResNet-50 and adds paths specifically to the keras_segmentation folder as well as the folder where the data generated from the application is stored.

```

35 # Base weights required by Keras to load before loading custom weights
36 # Puts the files in the users directory for Keras to find
37 base_weights = "resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5"
38 bundled_weights = os.path.join(base_path, ".keras", "datasets", base_weights)
39 bundled_json = os.path.join(base_path, ".keras", "keras.json")
40
41 keras_dir = os.path.join(os.path.expanduser("~"), ".keras")
42 datasets_dir = os.path.join(keras_dir, "datasets")
43 os.makedirs(keras_dir, exist_ok=True)
44 os.makedirs(datasets_dir, exist_ok=True)
45
46 if not os.path.exists(os.path.join(datasets_dir, base_weights)) and os.path.exists(bundled_weights):
47     copy(bundled_weights, os.path.join(datasets_dir, base_weights))
48
49 if not os.path.exists(os.path.join(keras_dir, "keras.json")) and os.path.exists(bundled_json):
50     copy(bundled_json, os.path.join(keras_dir, "keras.json"))

```

Figure 35: Paths to base weights and added directories for keras_segmentation folder and datasets

Through making these changes in the application's code, we were thus able to successfully extract the application for Dr. Thomas Huang's team to use.

Neural Network Training and Model Development

A primary objective of this work was to improve the neural networks used within the application, with a focus on increasing segmentation accuracy while maintaining efficient analysis time. The current implementation of the application is based on ResUNet-50 architecture, which combines a ResNet-50 encoder with a UNet style decoder. This architecture has served as a reliable baseline due to its relatively low computational cost, stable training behavior, and strong performance on moderate sized datasets.

ResUNet-50 is particularly effective in environments with limited computational resources as it contains significantly fewer parameters than many modern architectures, resulting in faster inference times and reduced training complexity [3]. Its convolutional structure enables efficient extraction of local spatial features, which are important for identifying embryo boundaries. However, this same reliance on convolutional operations limits its ability to capture long range spatial relationships within an image. As segmentation tasks become more complex, particularly when distinguishing finer biological structures, this limitation can lead to performance saturation. Additionally, convolutional models may be more susceptible to overfitting when trained on datasets that lack diversity.

Given these limitations and recent advancements in machine learning, alternative frameworks were explored to improve performance. The primary focus of this work was the implementation

of nnUNet, a self configuring framework designed to automatically adapt UNet based architectures to a given dataset [4]. Unlike traditional approaches that require manual selection of preprocessing steps, network depth, and training parameters, nnUNet analyzes dataset properties such as resolution, intensity distribution, and spatial characteristics to determine optimal configurations.

This level of automation allows nnUNet to consistently achieve strong performance across a wide range of medical image segmentation tasks. In benchmark studies, nnUNet has demonstrated state of the art results across multiple datasets without requiring manual architecture design [4]. The framework incorporates several techniques to improve robustness, including data augmentation, normalization, and five fold cross validation which helps reduce overfitting and improve generalization.

However, these advantages come with increased computational cost. nnUNet utilizes sliding window inference and test time augmentation, which require multiple passes over each image to generate predictions. While these techniques improve segmentation accuracy, they also increase inference time compared to simpler architectures such as ResUNet-50. As a result, nnUNet represents a trade off between computational efficiency and segmentation performance.

The implementation of nnUNet in this work required careful preparation of both the dataset and training environment. Input data had to be organized into a strict directory structure, and all images and labels were required to follow specific naming conventions. In addition, dataset specific configuration files were created to define input channels and segmentation classes. The training environment was set up using Anaconda based workflow, including installation of PyTorch with GPU support and configuration of the nnUNet framework. Throughout this process, various dependency and compatibility issues were resolved to ensure stable execution.

To support reproducibility and future development, the full workflow including environment setup, dataset preparation, preprocessing, training, and inference was documented in detail. This documentation allows future team members to replicate the training process and build upon the existing work without requiring extensive prior knowledge of the framework.

The primary metric used to evaluate model performance was Intersection over Union (IoU), which measures the overlap between predicted segmentation masks and ground truth labels relative to their union as seen in Equation 1 [5]. This metric produces values between zero and one with higher values indicating better segmentation accuracy.

$$IoU = \frac{\text{Intersection}}{\text{Ground Truth} \cup \text{Prediction Mask}}$$

Equation 1: Intersection over Union equation

In addition to IoU, Dice coefficient was used to further evaluate segmentation performance. The Dice coefficient emphasizes overlap between predicted and ground truth regions and is particularly useful in cases with class imbalance, which is common in medical imaging datasets. Compared to IoU, the Dice coefficient is more sensitive to difference in overlap and provides complementary insight into model performance [6].

$$Dice = \frac{2 \times \text{Intersection}}{\text{Ground Truth} \cup \text{Prediction Mask}}$$

Equation 2: Dice Coefficient equation

Inference time was also measured as a key performance metric, as practical deployment requires efficient processing of large numbers of images. While nnUNet generally requires more computation per image, its improved segmentation accuracy can justify the additional cost depending on application requirements.

Finally, Hausdorff distance was included as an additional metric to evaluate boundary accuracy. This metric measures the maximum deviation between predicted and ground truth segmentation boundaries. Although less central to this analysis it proves useful insight into edge level performance and may be valuable for future work focused on refining segmentation precision [7].

After completing dataset preparation, neural network models were trained using the nnUNet framework to perform segmentation of TE and ICM regions. nnUNet was selected due to its ability to automatically adapt network architecture and training parameters based on dataset characteristics, making it well suited for medical image segmentation tasks.

Initial training attempts, however, produced unsatisfactory results. The models exhibited low Dice coefficients, with values around 0.58, and validation loss increased over time rather than decreasing. This behavior suggested issues such as overfitting or inconsistencies within the dataset. Given the extent of the performance degradation, further investigation into the data pipeline was conducted.

Through this investigation, it was discovered that the primary issue was a misalignment between images and their corresponding label masks. This error was introduced during the dataset augmentation process, where transformations applied to images were not consistently applied to their associated labels. As a result, the model was effectively being trained on incorrect input output mappings, preventing it from learning meaningful segmentation patterns. To resolve this issue, the dataset preparation and augmentation process was repeated with careful verification at

each step to ensure that images and labels remained properly aligned. Once this correction was made, training was restarted and the results improved significantly.

Dataset Preparation and Preprocessing

A significant portion of this work focused on preparing high quality datasets for training neural networks to segment inner cell mass (ICM) and trophectoderm (TE) regions in embryo images. The success of any segmentation model is highly dependent on the quality and consistency of the training data, and early efforts in the semester revealed multiple issues that made the initially available datasets unsuitable for direct use.

The primary issue identified was related to the formatting and labeling of the datasets. The available ground truth label masks were stored as JPEG images and contained pixel intensity values ranging from 0 to 255, rather than discrete binary class labels. This introduced ambiguity in segmentation boundaries and made it difficult to reliably distinguish between regions of interest in the foreground and background regions. Initial attempts to resolve this issue through thresholding techniques were unsuccessful, as small variations in threshold values produced inconsistent and unreliable segmentation masks which can be seen in Figure 34. Given the sensitivity of neural networks to labeling accuracy, it was determined that these datasets were not suitable for training without further correction.

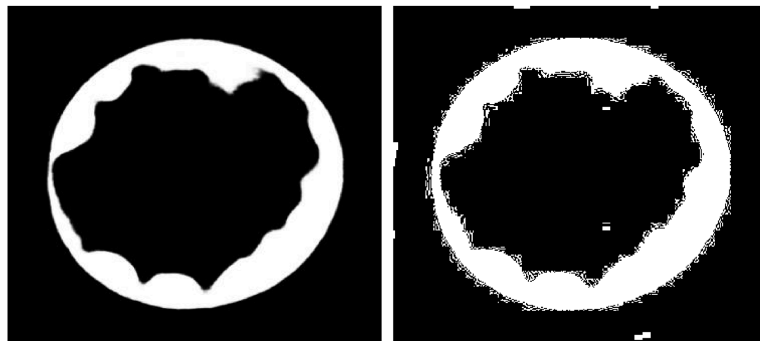


Figure 36: Conversion of Ground Truth from JPEG to PNG

To address this issue, the original dataset from an earlier project iteration in Spring 2022, was recovered. Unlike the JPEG datasets, the original files were stored in BMP format, which preserved the integrity of the segmentation labels. These labels correctly represented binary class information, making them suitable for use in supervised learning. A comprehensive preprocessing pipeline was then developed using Python to convert and standardize the dataset for compatibility with nnUNet.

This preprocessing pipeline includes multiple stages. First, all BMP images and corresponding labels were converted to PNG format to ensure compatibility with the training framework. Next, label values were normalized by converting all foreground pixel values from 255 to 1, resulting in strictly binary masks. Additional validation scripts were implemented to verify that each label file contained only the expected values, ensuring consistency across the dataset. Furthermore, all images were converted to grayscale to maintain uniformity in input channels, as inconsistencies between grayscale and RGB image can negatively impact model training.

In addition to correcting formatting issues, dataset augmentation was applied to improve model robustness and increase the effective dataset size. Each image was augmented through rotations and reflections, resulting in a significantly expanded dataset. Originally, the dataset included 249 images but after rotating each image in increments of 90 degrees and reflections of each, the dataset was increased to 1992 images and corresponding labels. This augmentation process helps the model generalize better by exposing it to variations in orientation and spatial configuration.

The dataset used for neural network training consisted of 1,992 labeled embryo images. A 78%/20%/2% split was used to divide the data into training, testing, and validation sets. This resulted in approximately 1,553 images (78%) for training, which were used to teach the neural network relevant features for embryo segmentation. The test set contained about 398 images (20%) and was used to evaluate model performance on unseen data to assess generalization capability. The remaining 40 images (2%) were assigned to the validation set and used during training to monitor performance and help reduce overfitting. This split ensured that most of the data contributed to learning while still reserving sufficient samples for unbiased evaluation and model tuning.

Another critical challenge encountered during preprocessing was the presence of inconsistent image dimensions. Many of the original images were rectangular and varied in size, which is incompatible with neural network training requirements. Furthermore, augmentation operations such as rotation introduced additional inconsistencies in image dimensions as seen in Figure 35. Several approaches were evaluated to address this issue, including cropping, stretching, and padding. Cropping was rejected due to the risk of removing important biological features, while stretching was deemed inappropriate because it could distort the structure of the embryo and negatively affect model learning.

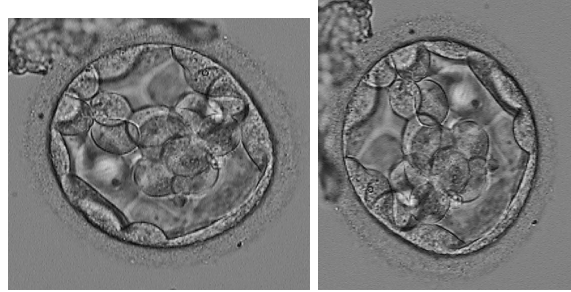


Figure 37: Example of Image Length and Width Inconsistency

The final solution involved implementing a reflection padding technique using OpenCV. This method extends the images by mirroring edge pixels, preserving the natural background texture and avoiding the introduction of artificial artifacts as seen in Figure 36. Reflection padding ensured that all images could be converted into square dimensions without losing meaningful information. Following this step, all images were resized to a standardized resolution of 512 x 512 pixels, ensuring compatibility with nnUNet input requirements.

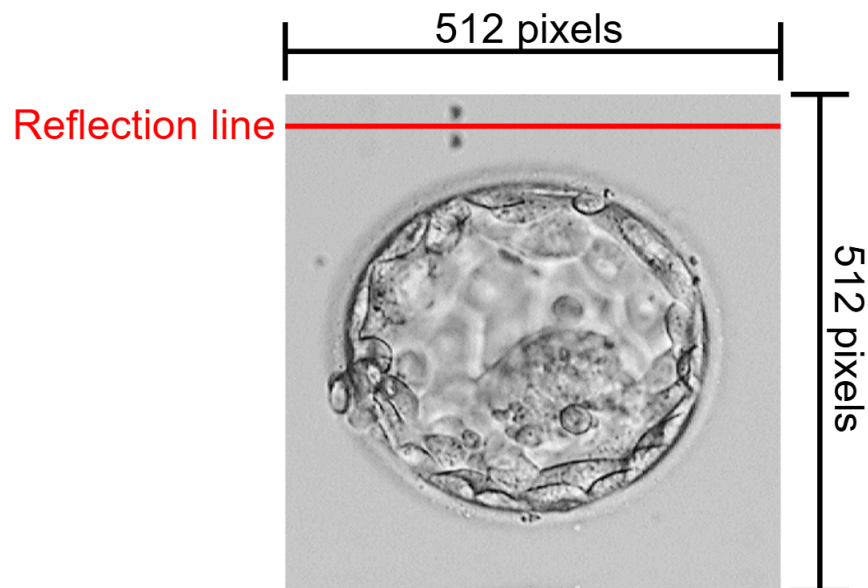


Figure 38: Reflection Padding Augmentation

Overall, the dataset preparation process was essential in establishing a reliable foundation for model training. By resolving formatting inconsistencies, correcting label structures, and standardizing image dimensions, a high quality dataset was produced that enabled effective neural network training.

Project Results:

Embryo Image Analysis Application Results

To evaluate the impact of the modular restructuring, the same videos were analyzed on both the previous codebase and the new modular application under identical hardware conditions. As shown in Figures 37 and 38, the per-step segmentation processing time in the old code fluctuated between approximately 833 ms and 1000 ms, with many steps reaching the full 1s/step ceiling and averaging roughly 960 to 970 ms per step. In the new modular application, the per-step time is consistently between approximately 749 ms and 805 ms, averaging roughly 765 to 770 ms per step. This represents an approximate reduction of 200 ms per step, resulting in roughly a 20 to 22% improvement in segmentation processing speed.

This speedup can be attributed to several structural improvements made during the refactoring process. The modular structure reduces overhead by ensuring each component only loads and executes what is relevant to its specific responsibility, rather than running through a large single-file script where all logic is entangled. Redundant functions were removed, eliminating operations during runtime. Additionally, replacing the three parallel global lists used to track video data with a single dictionary improved data lookup efficiency, as dictionaries retrieve values directly by key rather than requiring the application to search through the index into multiple lists simultaneously. Altogether, these changes produced a measurably faster and more consistent analysis runtime that will benefit users processing large numbers of videos.

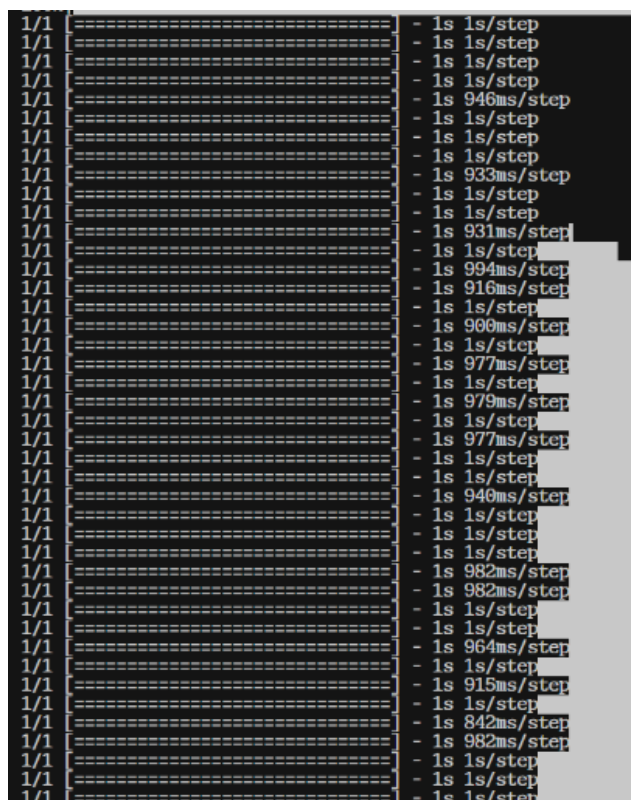


Figure 39: index-21_ctk analyzing time

```

1/1 [=====] - 1s 771ms/step
1/1 [=====] - 1s 762ms/step
1/1 [=====] - 1s 772ms/step
1/1 [=====] - 1s 781ms/step
1/1 [=====] - 1s 772ms/step
1/1 [=====] - 1s 760ms/step
1/1 [=====] - 1s 761ms/step
1/1 [=====] - 1s 767ms/step
1/1 [=====] - 1s 755ms/step
1/1 [=====] - 1s 749ms/step
1/1 [=====] - 1s 775ms/step
1/1 [=====] - 1s 805ms/step
1/1 [=====] - 1s 782ms/step
1/1 [=====] - 1s 776ms/step
1/1 [=====] - 1s 763ms/step
1/1 [=====] - 1s 774ms/step
1/1 [=====] - 1s 758ms/step
1/1 [=====] - 1s 765ms/step
1/1 [=====] - 1s 758ms/step
1/1 [=====] - 1s 755ms/step
1/1 [=====] - 1s 756ms/step
1/1 [=====] - 1s 758ms/step
1/1 [=====] - 1s 774ms/step
1/1 [=====] - 1s 762ms/step
1/1 [=====] - 1s 762ms/step
1/1 [=====] - 1s 760ms/step
1/1 [=====] - 1s 764ms/step
1/1 [=====] - 1s 777ms/step
1/1 [=====] - 1s 774ms/step
1/1 [=====] - 1s 766ms/step
1/1 [=====] - 1s 790ms/step
1/1 [=====] - 1s 780ms/step
1/1 [=====] - 1s 759ms/step

```

Figure 40: Modular code runtime

Another observable behavior during analysis is the OCR fallback and timestamp correction system within the `_extract_frames` method, as shown in Figure 39. When Tesseract OCR fails to read a timestamp from a video frame – which can occur due to image noise, low contrast, or motion blur at that region of the frame, the application does not crash or skip the video entirely. Instead, it falls back to the previously successfully read timestamp value and continues processing. This ensures that a small number of unreadable frames does not compromise the entire analysis of a video.

Additionally, the output shows instances where the application detected and corrected misread timestamps, such as correcting a value from 42.2 to 842.2 and from 42.5 to 842.5 as shown in Figure 39. These corrections occur when the OCR reads a timestamp that is inconsistent with the progression of previous frames. For example, a four-digit timestamp such as 842.2 can be misread as a three-digit value of 42.2 because the leading digit was not captured. The application identifies this discrepancy by comparing the current reading against the previous extracted timestamp and, if the difference is illogically large or the digit count drops unexpectedly, it reconstructs the correct value by attaching the missing leading digit to the front of the misread value. This self-correcting behavior improves the accuracy and reliability of the growth rate calculations, since timestamps directly determine the time axis of the embryo size data used in the linear regression. This fallback and self-correction system ensures that minor OCR failures on individual frames do not invalidate an entire video's analysis, making the application

significantly more robust and reliable when processing real-world embryo videos where image quality is not always consistent.

```
OCR failed on frame 22, using fallback: 192
OCR failed on frame 24, using fallback: 98
OCR failed on frame 25, using fallback: 98
OCR failed on frame 26, using fallback: 98
OCR failed on frame 29, using fallback: 215
OCR failed on frame 34, using fallback: 2
OCR failed on frame 35, using fallback: 2
OCR failed on frame 38, using fallback: 45
OCR failed on frame 43, using fallback: 2
OCR failed on frame 44, using fallback: 2
OCR failed on frame 47, using fallback: 75
OCR failed on frame 49, using fallback: 2
OCR failed on frame 50, using fallback: 2
OCR failed on frame 53, using fallback: 5
OCR failed on frame 54, using fallback: 5
OCR failed on frame 68, using fallback: 45
OCR failed on frame 73, using fallback: 2
OCR failed on frame 74, using fallback: 2
OCR failed on frame 77, using fallback: 275
OCR failed on frame 79, using fallback: 2
OCR failed on frame 84, using fallback: 339
OCR failed on frame 87, using fallback: 9
OCR failed on frame 89, using fallback: 8415
Corrected misread timestamp from 42.2 to 842.2
Corrected misread timestamp from 42.5 to 842.5
```

Figure 41: Detected misread timestamps

Inner Cell Mass and Trophectoderm Test Results

The TE segmentation model demonstrated strong convergence behavior, with Dice coefficients approaching 1.0. Quantitative evaluation on the test dataset yielded a mean Dice score of 0.9989 with a very low standard deviation of 0.0012, indicating both high accuracy and consistency across samples. The intersection over union (IoU) was 0.9978 and the average Hausdorff distance was 1.105 pixels. In addition to accuracy improvements, inference time was measured at approximately 0.438 seconds per image, representing efficient performance suitable for practical applications.

The ICM model also achieved strong performance, with a Dice score of 0.9996. While some variability in training metrics was observed during earlier epochs seen in Figure 37, this behavior is common in patch-based training approaches where certain regions of the image may be more difficult to segment. As training progressed and the learning rate decreased, these fluctuations diminished, and the model converged to a stable solution. Other quantitative evaluations included an intersection over union of 0.9991 and an average Hausdorff distance of 1.386 pixels. Inference time was measured at approximately 0.496 seconds per image.

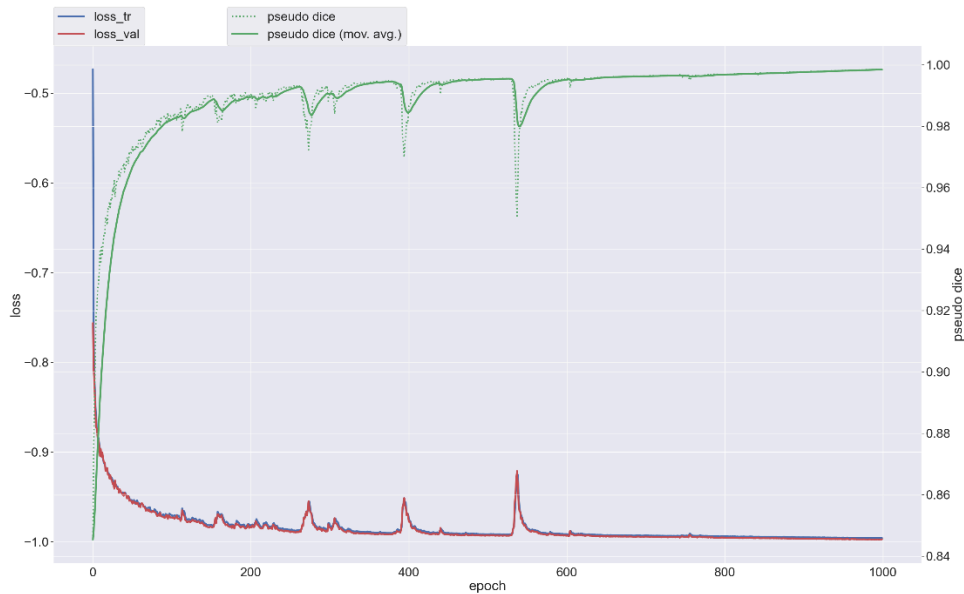


Figure 42: Inner Cell Mass Training Performance per Epoch

These results highlight the importance of proper dataset preparation in neural network training. Once data inconsistencies were resolved, nnUNet was able to achieve highly accurate segmentation performance for both TE and ICM tasks. The models demonstrated strong generalization within the dataset and produced reliable segmentation outputs.

	Mean Dice	Dice Standard Deviation	Inference Time (s) / Image	Intersection over Union (IoU)	Average Hausdorff Distance (pixels)
Trophectoderm	0.9989	0.0012	0.434	0.9978	1.105
Inner cell mass	0.9996	0.0003	0.496	0.9991	1.386

Table 1: nnUNet Inner Cell Mass and Trophectoderm Performance Metrics

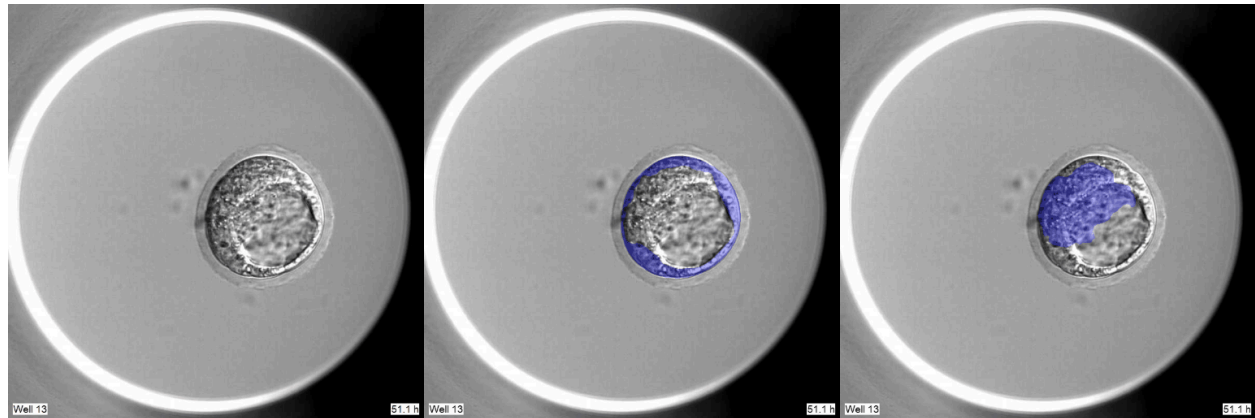
Inference Pipeline and Performance Evaluation

In addition to model training, a complete inference pipeline was developed to extend segmentation capabilities to video data. This pipeline enables the application to process video embryo data by performing segmentation on individual frames extracted from videos. The inference process begins by decomposing a video into its constituent frames. Each frame is then preprocessed to match the format used during training, including conversion to grayscale and resizing to the standardized resolution of 512 x 512 pixels. Proper file naming conventions are applied to ensure compatibility with nnUNet's inference requirements. Once preprocessing is complete, the trained models are used to generate segmentation masks for each frame.

Following inference, the predicted segmentation masks are overlaid onto the original frames to allow for qualitative evaluation. These frames can then be recombined into animated format such as GIFs, to visualize segmentation results over time. There were no ground truth maps corresponding to the videos and therefore there is no way to get a quantitative value determining accuracy. This approach provides a practical method for analyzing embryo development in a dynamic context.

Two videos were used to evaluate the inference pipeline and examine both inner cell mass and trophectoderm segmentation outputs. For each cell structure, two videos were used, and both videos within each pair corresponded to the same embryo sequence. However, despite being identical inputs within each pair, the visual appearance of the embryo over time varied significantly between the two cases, which allowed for a clearer assessment of model robustness under different developmental patterns.

In the first set of videos, the embryo progression appeared relatively consistent with types of images used during training. The overall structure remained stable across frames, and changes in shape and gradual and continuous rather than abrupt. In this scenario, both models produced more visually coherent segmentation masks when compared to the second video set. While some inaccuracies were still present, particularly at finer boundary regions, the segmentation output generally followed the expected structure of the embryo over time. The trophectoderm model in particular maintained a more stable outer boundary, with fewer abrupt changes in segmentation shape between frames. The ICM model had more variation in the central region but struggled to track the general location of the inner structure when the embryo appearance remained similar to the trained distribution. Towards the end of the video, the ICM model also had false positives along the corners of the images.



Original

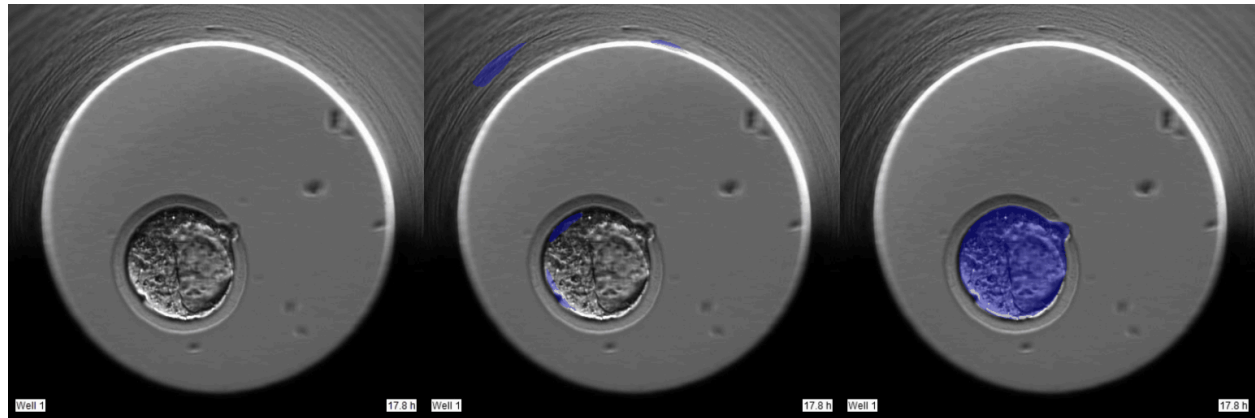
Trophoderm

Inner Cell Mass

Figure 43: First Video Set ICM and TE Comparison

In contrast, the second video in each pair exhibited a noticeably different visual progression. In these cases, the embryo structure changed in a way that was less consistent with previously seen training examples as seen in Figure 39. The most prominent difference was observed in the central region of the embryo, which appeared to shift or expand in a manner that was not commonly represented in the training dataset. At certain points in the sequence, the embryo appearance gave the impression of a transition in structure that made segmentation more challenging to maintain consistently across frames. Although this is purely a visual observation from the model's input data, it highlights a clear difference in distribution compared to the more standard cases.

Under these conditions, both models showed a decline in performance. The segmentation masks were less stable with increased variability between consecutive frames. This was especially noticeable in the ICM outputs, where the predicted inner region often fluctuated or became less clearly defined as the video progressed. The TE model also experienced some less accurate predictions but remained comparatively more stable, particularly in maintaining a recognizable outer boundary of the embryo across frames.



Original

Trophectoderm

Inner Cell Mass

Figure 44: Second Video Set ICM and TE Comparison

Despite strong performance on training dataset, testing on the training dataset, testing on new video revealed limitations in model generalization. The models exhibit false positives and false negatives, particularly on regions near image boundaries and in frames that differed significantly from the training data. These discrepancies were attributed to differences in image characteristics, such as lighting conditions, noise patterns, and embryo appearance.

The results demonstrate that neural network performance is highly dependent on similarity between training and inference data. Even highly accurate models can experience performance degradation when applied to data outside their training distribution. It was observed that the TE model generally performed better than the ICM model under these conditions, although both were affected by distribution shifts.

These findings emphasize the importance of maintaining consistent data collection and preprocessing procedures when deploying machine learning models in real world applications. Future improvements may include expanding the training dataset to include more diverse samples or implementing domain adaptation techniques to improve robustness.

Overall, the developed inference pipeline provides a functional bridge between model training and practical application. While further work is needed to improve generalization, the current system demonstrates the feasibility of applying neural network based segmentation to video data in the context of embryo analysis.

Light Detection and Ranging (LiDAR) Neural Networks

A new problem that the Image Analysis Team began to tackle this semester is aiding MicroVIP's light detection and ranging (LiDAR) team in their goal to produce an autonomous system to detect unexploded ordnance, also known as UXOs. This umbrella term encompasses any ammunition that did not originally detonate upon firing. UXOs are a known issue that plague prior combat and military training sites around the world. More specifically, the LiDAR team is targeting UXOs that can be found locally within the Hawaiian islands, such as Makua Valley on Oahu, the island of Kaho'olawe, as well as large portions of both military and civilian land on the island of Hawai'i. LiDAR systems have the advantage of gathering information on the depth and reflectance of objects within a point cloud, making it potentially useful for identifying UXOs hidden within uneven terrain. Due to the Image Analysis Team's prior experience with neural networks and instance segmentation, we were tasked with training NNs that can automatically detect UXOs for removal.

Candidate Neural Networks

We first identified existing neural network architectures that would be able to process the three-dimensional point clouds which are the raw data that LiDAR systems capture. After doing a small literature review, we decided on testing two neural networks that we deemed the most suited for this task.

The first architecture we chose was Point-Voxel Region-based Convolutional Neural Network, also known as PV-RCNN [8]. PV-RCNN utilizes both the raw point cloud of the data and the resulting voxels, or three-dimensional images composed of three-dimensional pixels based on the point cloud data. After transforming the point cloud to voxel form, the voxel form undergoes three-dimensional sparse convolution and is then converted to a two-dimensional bird's eye view to undergo basic classification. Meanwhile, the point cloud is used to sample keypoints. The information from both processes is pooled together to identify regions of interest (RoIs) and thus generate three-dimensional classification boxes. Through extracting information from both representations of the data, PV-RCNN has a high propensity towards identifying irregular or potentially occluded objects from LiDAR data, making it suitable for UXOs that may be hidden within vegetation or rocky terrain. The team intends on using the latest version of PV-RCNN that is publicly available, known as PV-RCNN++, the architecture of which is featured in Figure 42.

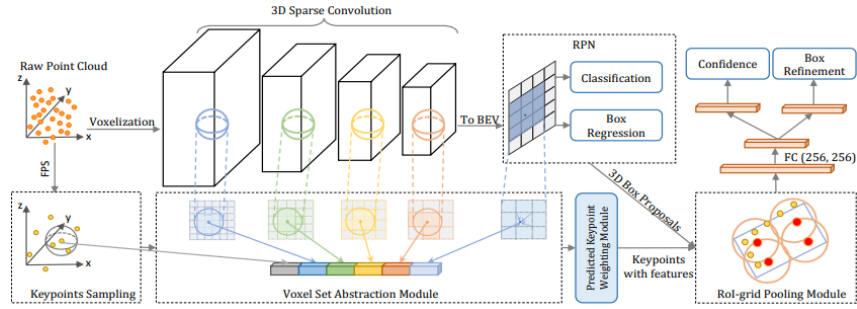


Figure 45: Model architecture of PV-RCNN++ [PV-RCNN++]

The second LiDAR machine learning architecture the team chose to investigate was Voxel R-CNN [9] which stands for Voxel Region-based Convolutional Neural Network. Unlike the PV-RCNN models, Voxel R-CNN is a purely voxel-based framework for 3D object detection. The architecture consists of two key stages. It first performs deep learning on the voxel-only representation using three-dimensional convolutions and then flattens the image to a bird's eye view to be better processed by a two-dimensional backbone network. The outputs are then pooled to generate regions of interest. This architecture is then featured in Figure 43 below.

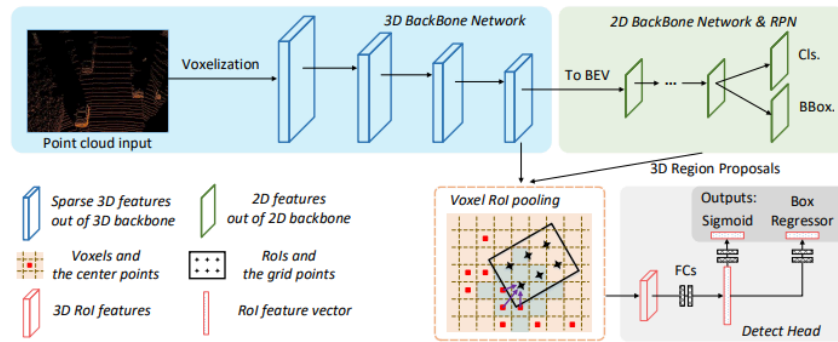


Figure 46: Model architecture of Voxel R-CNN [Voxel]

Both the PV-RCNN models and Voxel R-CNN are amongst the highest performing LiDAR neural networks that are available with current research [8, 9] and are sufficiently different for us to test and understand different approaches to interpreting LiDAR data. Another reason why the two architectures were chosen was due to their availability within the publicly available Python toolbox Open source Point Cloud Detection, also known as OpenPCDet [10]. This toolbox supports training and testing for a variety of high-performance LiDAR neural networks. By using the functionalities of a singular toolbox, this increases the efficiency of training and

testing the two machine learning architectures we selected through the use of shared and similar Python scripts. This also holds the advantage of only needing to concern ourselves with the dependencies of a singular environment rather than a different environment for each neural network, a challenge we faced with the neural network architectures we experimented with in past semesters.

When setting up OpenPCDet, we ran into issues with the environment required of the toolbox, specifically in regards to the sparse convolution Python library, `spconv` that is utilized [11]. OpenPCDet has only been officially tested on Linux operating systems, whereas both the SiGE and GaN lab computers have Windows 10 or 11 operating systems, respectively. The laptops and personal computers of all Image Analysis Team members run only on Windows 11 or the Mac operating system. This was a significant issue as the `spconv` library is known to be “picky” about having a Linux operating system, certain dependencies, and a CUDA 11.4 or CUDA 11.8 driver.

To work around the Linux operating system and particular dependencies of the toolbox, the team decided to run an isolated Linux operation system and environment on the GaN lab computer. This would aid in reducing the possibility of conflicts with the dependencies, allow multiple team members to partake in training and testing the neural networks, and decrease computation time due to the high-performance specifications of GaN.

To create this environment, we utilized a program called Docker. Docker is a program that is used to build and run containers [12]. Containers are used to simulate operating systems without simulating the entire computer. It essentially acts as a sand boxed environment to host operating systems and environments isolated from the actual operating system of the computer it is running on while being a “lighter” method of doing so compared to similar software such as Virtual Machine. Containers are easily transferable across computers, making Docker an ideal choice for a member of the team to create the container for OpenPCDet on their personal computer, then copy it over to a lab computer for the rest of the team to utilize in training and testing.

The team installed Docker on the GaN lab computer in anticipation of training. Throughout the semester, due to limitations in other parts of the LiDAR efforts as well as the fact that OpenPCDet has been tested with only CUDA driver 11.3 while the GaN computer has CUDA 13.0 for nnUNet training, training has not yet begun for either of the machine learning models we selected to train on our LiDAR data. At the time of writing this final report, the CVE-2026-31431 vulnerability, also known as “Copy Fail”, was found within Linux kernels [13]. This has been classified as a high-severity vulnerability as it enables unprivileged users to gain root access to all major Linux distributions. The version of Linux that the team installed on GaN to run Docker, Windows Subsystem for Linux, also known as WSL2, as well as Docker, have been removed from GaN out of an abundance of caution in regards to this vulnerability. As such, only once this fatal vulnerability has been patched and there is sufficient LiDAR data collected by the team to begin training the neural networks should WSL2 and Docker be

reinstalled. This can be done through following basic online documentation from both the Docker and Windows Subsystem for Linux development teams.

Training Data for LiDAR Neural Networks

As the Image Analysis team is utilizing pre-existing architectures for the training of the LiDAR neural networks we have selected, we must tailor our datasets to accommodate the point clouds that the LiDAR team will produce. While waiting for the custom point clouds from LiDAR dataset, we created a few methodologies to enhance the robustness of the eventual training datasets and thus the robustness of our neural networks.

First and foremost, we needed a low-cost solution to label the UXOs and other objects within the point clouds. As University of Hawai'i students have access to Matlab licenses, we found that the LiDAR Labeler app in Matlab would be a good solution to this problem [14]. This allows the user to load, view, and label point cloud data, even including a feature to automatically label some of the data, aiding in reducing the time necessary for this part of the project.

As for the datasets themselves, we understand from experience with the training of the ICM and TE neural networks that using data collected from the same set up that will be used in the real-world implementation aids significantly in the transferability of the neural network's performance between experimental testing on a computer and in the field deployment. Meanwhile, one of the main limitations of utilizing a purely custom dataset based solely upon the data collected by the LiDAR team would be the amount of data there is to train, validate, and test the neural networks. To find a compromise between the two extremes, we decided to look for online, publicly available LiDAR datasets of Hawaiian terrain and vegetation. To these datasets we would artificially inject custom scans of UXOs from the LiDAR team. This would aid significantly not only in expanding the datasets but also getting the neural networks accustomed to identifying UXOs in a variety of terrain.

Building upon this idea of artificially injecting point clouds of UXOs into already existing LiDAR scans, UXO detection in the field would involve partially or even heavily occluded UXOs. LiDAR data, also for other applications such as autonomous driving, often have to deal with shadowing, or occlusion of point cloud data from obstacles in front of the target object. This is also a challenge in the field of UXO detection, especially in Hawaii where a lot of the terrain is heavy with vegetation and rocky, uneven surfaces [15]. LiDAR data may also suffer from partial or incomplete point cloud data of objects if the scan is incomplete or not detailed enough. To best simulate these real-world problems with LiDAR detection, we would include artificial occlusions of point cloud data. This could be done by removing a random set of up to 70% of points associated with a scan of a UXO. Scripts to automatically simulate the effects of shadowing and other known phenomena within LiDAR scans would also aid in improving the robustness of the datasets available to us. In addition to these more sophisticated techniques, simple point cloud

software such as LiDAR Labeler mentioned above can also be utilized to perform basic duplications, rotations, and translations of objects within LiDAR data.

While waiting for the LiDAR UXO scans, the team inspected scans from publicly available databases, most notably, the United States' Geological Survey (USGS) database [15]. These publicly available scans were from aerial LiDAR data as part of mapping out the elevation of Hawaii's terrain. As a result, the need for high-density point clouds in these databases was nonexistent, so very few have the point cloud density necessary for visualizing vegetation and rocky terrain from the ground level. Figure USGS! below is a sample image from an aerial scan of volcanic terrain on Hawaii island and is noted to have a point cloud density specification of QL 0 and source digital elevation model (DEM) of 0.25 meters, meaning that for the purposes of terrain mapping, it meets the highest accuracy standard defined by the USGS specifications. Despite this, upon comparison between this best LiDAR aerial scan we were able to obtain and sample data from the LiDAR team [15], it was clear that the point cloud density between the two systems was too different, making the injection of UXO data into publicly available LiDAR datasets a less than ideal method of training data expansion. The Image Analysis team instead intends on working with the LiDAR team to collect as many diverse scans as possible to label and use for training and testing purposes.

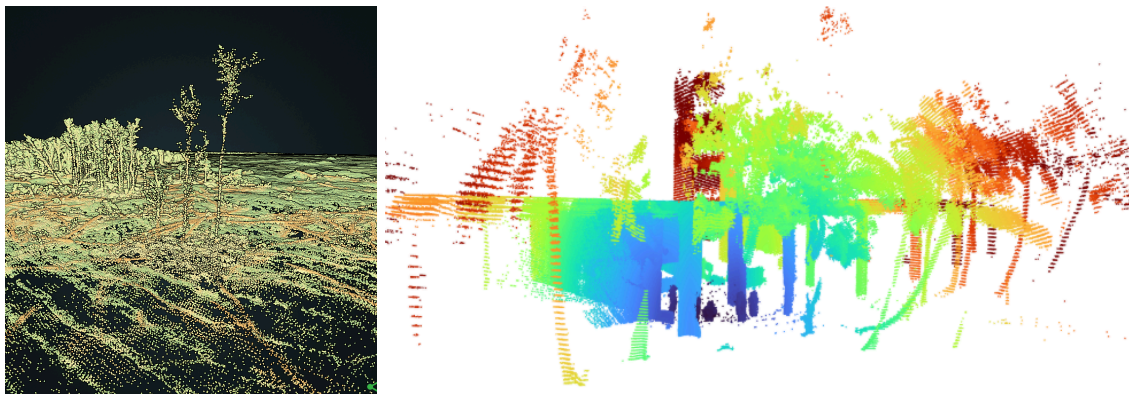


Figure 47: Sample aerial LiDAR scan of Hawai'i Island (left) and sample LiDAR scan from terrestrial scanner (right)

Due to both the LiDAR system development and LiDAR neural networks being developed in parallel, the Image Analysis team is still awaiting viable data from the LiDAR team to begin the data labeling process. Nevertheless, this semester's research has laid the foundation for future teams to build off of.

Project Budget:

Software & Tools	Price
Tower Desktop (AMD Ryzen 7 CPU & Nvidia RTX 3080 GPU; Not including peripherals)	\$1633
Alienware Area-51 Gaming Desktop (Intel Core Ultra 7 265K processor, NVIDIA GeForce RTX 5090 GPU, 32 GB RAM; Not including peripherals)	\$4750
Software in MATLAB	\$860/year
Processing Image Toolbox	\$400/year
Deep Learning Toolbox	\$500/year
Python 3.9.19	\$0
TensorFlow	\$0
Pytorch	\$0
Ultralytics (YOLOv8)	\$0
Total Cost:	\$8143

Table 2 : Software and Tools Plus Cost Used

Conclusion:

To summarize our final report, the Embryo Image Analysis team made significant progress this semester, through updating the user interface and application software, training the new nnUNet neural network model with refitted datasets, and implementing LiDAR technology to detect and locate UXOs. Notably, the application software was updated by clarifying and simplifying functions and methods in Github, so now the code is easier to read and understand. This also means that the user interface has a more modern look and is easier for users to utilize. The new nnUNet model performs slightly better compared to the old ResUNet-50 model, with shorter time per image, and a satisfactory pseudo dice value. As for working with the LiDAR team, we found two neural networks capable of processing the 3D point-cloud data that the LiDAR systems capture and have created a strong framework for future semesters to build off of.

Acknowledgements:

We would like to thank Dr. Thomas Huang and his clinic for their collaboration with our project. We would also like to express our gratitude for Dr. Aaron Ohta for his continued support and guidance. As a nod to our fellow MicroVIP members, we'd like to express thanks to the LiDAR team for their collaboration throughout this past semester.

References:

- [1] Python Software Foundation, “Classes,” *Python 3 Documentation*, 2024. [Online]. <https://www.python.org/doc/>
- [3] S. Saifullah, R. Drezewski, A. Yudhana, and A. P. Suryotomo, “Automatic Brain Tumor Segmentation: Advancing U-Net With ResNet50 Encoder for Precise Medical Image Analysis,” *IEEE Access*, vol. PP, no. 99, pp. 1–1, 2025, doi: 10.1109/ACCESS.2025.3547430.
- [4] F. Isensee, J. Petersen, A. Klein, D. Zimmerer, P. F. Jaeger, S. Kohl, J. Wasserthal, G. Koehler, T. Norajitra, S. Wirkert, and K. H. Maier-Hein, “nnU-Net: Self-adapting Framework for U-Net-Based Medical Image Segmentation,” *arXiv*, Sep. 2018.
- [5] A. Rosebrock, “Intersection over Union (IoU) for object detection,” [PyImageSearch](https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/), Nov. 7, 2016. [Online]. Available: <https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/>. [Accessed: May 9, 2026].
- [6] Lathashree Harisha, “Dice Coefficient! What is it? - Lathashree Harisha - Medium,” *Medium*, Feb. 19, 2023.
- [7] “Hausdorff Distance — skimage 0.25.2 documentation,” *Scikit-image.org*, 2025. https://scikit-image.org/docs/0.25.x/auto_examples/segmentation/plot_hausdorff_distance.html
- [8] S. Shi *et al.*, “PV-RCNN++: Point-Voxel Feature Set Abstraction With Local Vector Representation for 3D Object Detection,” *arXiv (Cornell University)*, Jan. 2021, doi: <https://doi.org/10.48550/arxiv.2102.00463>.
- [9] J. Deng, S. Shi, P. Li, W. Zhou, Y. Zhang, and H. Li, “Voxel R-CNN: Towards High Performance Voxel-based 3D Object Detection,” *arXiv (Cornell University)*, Jan. 2020, doi: <https://doi.org/10.48550/arxiv.2012.15712>.
- [10] open-mmlab, “GitHub - open-mmlab/OpenPCDet: OpenPCDet Toolbox for LiDAR-based 3D Object Detection,” *GitHub*, 2025. <https://github.com/open-mmlab/OpenPCDet>
- [11] Y. Yuan, “Setting up environment for the open source 3D Object Detection project OpenPCDet using Docker,” *Medium*, Sep. 18, 2020. <https://yunshuang.medium.com/setting-up-running-environment-for-the-open-source-3d-object-detection-project-openpcdet-using-7427c0091edd> (accessed May 06, 2026).

- [12] Docker, “Enterprise Application Container Platform | Docker,” *Docker*, 2024. <https://www.docker.com/>
- [13] R. Team, “CVE-2026-31431: Copy Fail vulnerability enables Linux root privilege escalation across cloud environments | Microsoft Security Blog,” *Microsoft Security Blog*, May 02, 2026. <https://www.microsoft.com/en-us/security/blog/2026/05/01/cve-2026-31431-copy-fail-vulnerability-enables-linux-root-privilege-escalation/>
- [14] “Get Started with the Lidar Labeler - MATLAB & Simulink,” *Mathworks.com*, 2026. <https://www.mathworks.com/help/lidar/ug/lidar-labeler-get-started.html> (accessed May 06, 2026).
- [15] USGS, “USGS.gov | Science for a changing world,” *Usgs.gov*, 2019. <https://www.usgs.gov/>

